Chapitre I-II
Virtualisation
Software-Defined Networking

MOHAMMED BOUHORMA





Sujets du cours :

- Introduction au SDN et à la softwarisation des réseaux
- Fondamentaux d'OpenFlow et des contrôleurs SDN
- Virtualisation de réseau et réseaux superposés
- Cas d'utilisation du SDN dans les réseaux de centres de données et le cloud computing
- Sécurité et gestion du SDN
- Tendances futures du SDN et de la softwarisation des réseaux

Travaux pratiques : Tout au long de ce cours, les étudiants travailleront sur différents travaux pratiques couvrant
Les concepts fondamentaux du SDN et de la softwarisation des réseaux, y compris le protocole OpenFlow, les contrôleurs SDN et la virtualisation de réseau.
De plus, les travaux pratiques se concentreront sur l'application du SDN et de la softwarisation des réseaux dans des scénarios réels tels que les réseaux de centres de données et le cloud computing.
En réalisant ces travaux pratiques, les étudiants acquerront des compétences pratiques en configuration de topologies réseau, commutateurs virtuels et routeurs, et en test de la performance du réseau.

TP & Ateliers

Introduction à Mininet et création d'une topologie réseau simple
Objectif : Simuler une topologie basique avec Mininet, comprenant des hôtes et des commutateurs, et tester la connectivité réseau.

Contrôle de flux réseau avec OpenFlow
Objectif : Configurer et gérer des flux réseau en utilisant OpenFlow et un contrôleur SDN (comme OpenDaylight, ONOS), pour comprendre comment SDN sépare le plan de contrôle du plan de données.

Équilibrage de Charges Dynamique avec OpenFlow, ONOS et OpenDaylight

Objectif : Mettre en place un mécanisme d'équilibrage de charge pour répartir dynamiquement le trafic entre plusieurs serveurs en fonction des règles.

Contrôle de Flux Dynamique en Temps Réel avec OpenFlow

Objectif : Modifier dynamiquement les règles de flux pour gérer le trafic en fonction des conditions du réseau (ex. surcharge, latence élevée).

Configuration Automatique des Commutateurs via OpenDaylight, ONOS

Objectif : Démontrer comment OpenDaylight, ONOS peut automatiquement configurer des commutateurs dans un réseau SDN.

Où doit vivre l'intelligence du réseau ?

Nous répondrons à cette même question trois fois — et chaque réponse déplacera le problème.

- **A** Dans chaque boîte
- **B** Dans les protocoles
- **C** Dans une vue globale



Le réseau est déjà « intelligent ». Mais intelligent pour quoi ?

Question-pivot Gardez votre réponse. Nous la testerons contre la réalité.

Le constat qui dérange

QUESTION-PIVOT

« Le réseau sait calculer une route. Il ne sait pas ce que vous voulez. »

INTENTION MÉTIER

« Isoler Finance, préserver DNS, prioriser la visioconférence. »

Une intention — exprimée en langage de service.

traduction humaine

RÉALITÉ OPÉRATIONNELLE

ACL | VLAN | QoS | BGP | NAT | VRF | FW

Une intention devient une collection de configurations locales, distribuées et dépendantes de l'ordre.

La manip « à l'ancienne », littéralement

```
SW-23 — Cisco IOS / IOS-XE

SW-23# configure terminal
SW-23(config)# ip access-list extended QUARANTINE
SW-23(config-ext-nacl)# permit udp 10.30.0.0 0.0.255.255 any eq domain
SW-23(config-ext-nacl)# permit tcp 10.30.0.0 0.0.255.255 any eq domain
SW-23(config-ext-nacl)# deny ip 10.30.0.0 0.0.255.255 any
SW-23(config-ext-nacl)# permit ip any any
SW-23(config)# interface Vlan30
SW-23(config-if)# ip access-group QUARANTINE in
SW-23(config-if)# end
SW-23# write memory
```

Nuance : appliquée sur l'interface VLAN, cette ACL contrôle le trafic routé depuis VLAN 30 ; elle ne stoppe pas à elle seule les échanges L2 internes au VLAN.



10 lignes × 48 boîtes

≈ 480 commandes à saisir, vérifier et sauvegarder

La syntaxe est exacte. Le problème est le modèle opératoire.

Puis l'échelle transforme une erreur en incident

ÉCHELLE × ORDRE × FILET

48 équipements

Sessions, syntaxes, versions et contextes différents.



7 étapes ordonnées

Une ACL posée avant la route de service peut couper le réseau.



0 transaction globale

À mi-déploiement, le réseau peut être dans un état hybride.



Automatiser la saisie n'est pas encore changer de modèle.

Ansible, NETCONF/YANG ou des scripts réduisent le travail manuel. Ils ne garantissent pas, à eux seuls, une intention globale ni un commit transactionnel multivendeur.

Et si le réseau recevait l'intention une seule fois ?

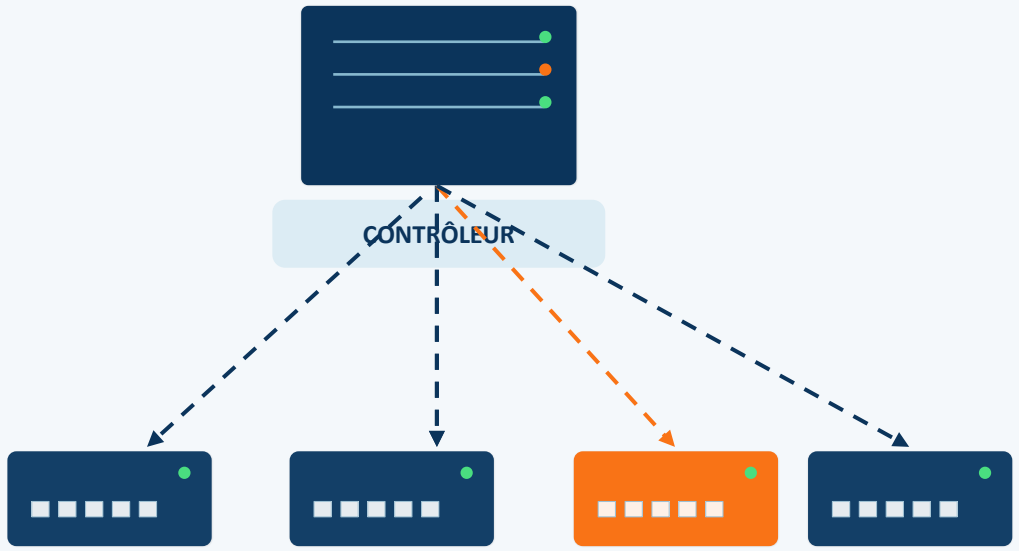
INTENTION — PSEUDO-CODE

```
policy quarantine-vlan30 {
  allow dns;
  redirect web -> fw-cluster;
  deny internal;
}
```

Déclarer le résultat souhaité — pas la suite de commandes.

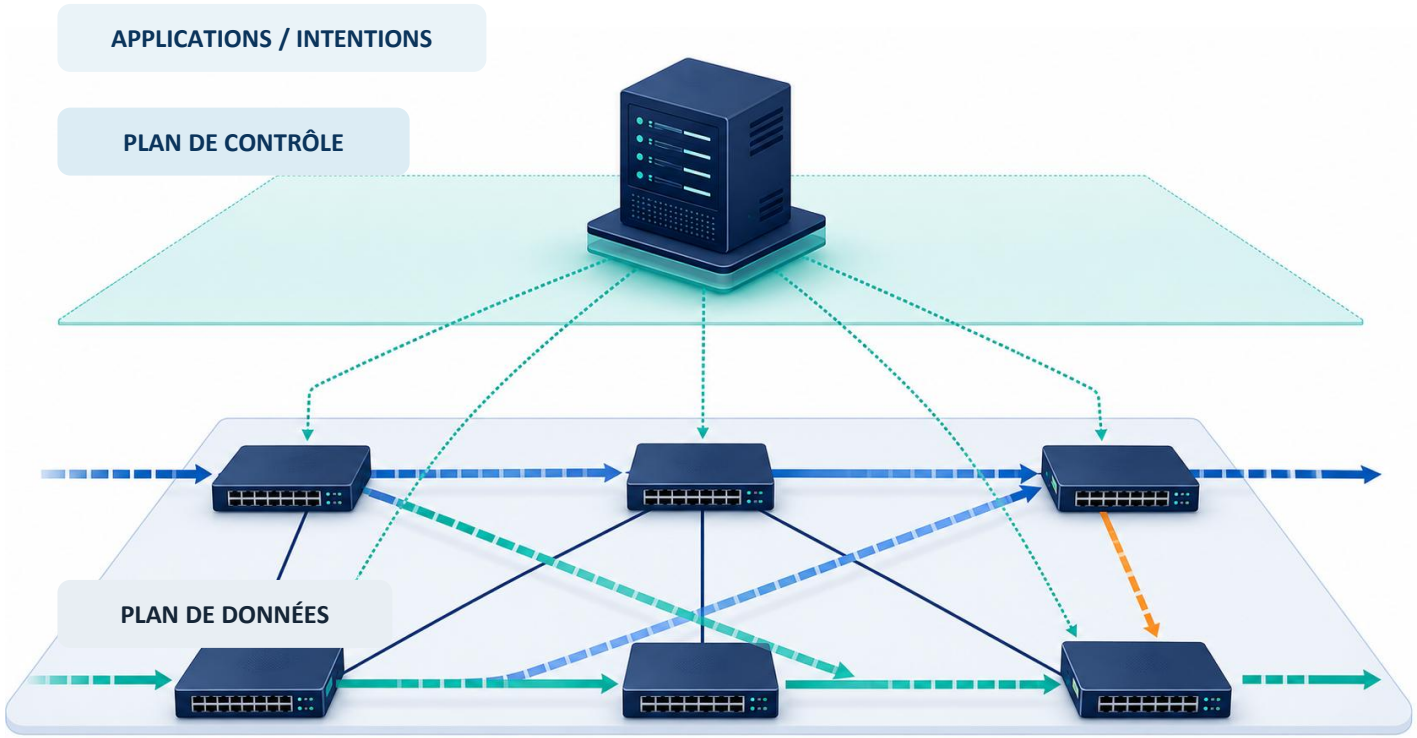
compiler

UNE VUE GLOBALE



Le contrôleur traduit la politique en règles cohérentes pour les équipements concernés.

La révélation : séparer la décision du transfert



SDN

Une architecture où le comportement du réseau devient programmable à partir d'une logique de contrôle globale.

- **Logiquement centralisé**
  Le contrôle peut être répliqué et distribué physiquement.
- **OpenFlow est une option**
  Un protocole southbound pour programmer des tables de flux — pas la définition complète du SDN.
- **Commutateurs ≠ stupides**
  Ils exécutent le forwarding à haute vitesse, avec des pipelines et capacités propres.

Rebouclage : où doit vivre l'intelligence ?

Au-dessus des équipements, dans une logique globale — mais répliquée, observable et contrôlée.

La question suivante n'est donc plus « quel CLI ? » **mais « quel modèle de contrôle ? »**



Prochaine étape : ouvrir un commutateur OpenFlow et regarder ce qu'il sait réellement faire.

SDN

Et si le vrai problème n'était pas le réseau…
mais la façon de le piloter ?

Une introduction progressive au Software-Defined Networking

QUESTION-PIVOT

> **Si vous aviez 10 000 équipements, les configureriez-vous un par un ?**



Le modèle TCP/IP

Une architecture en 4 couches pour comprendre comment les données circulent sur un réseau et sur Internet.

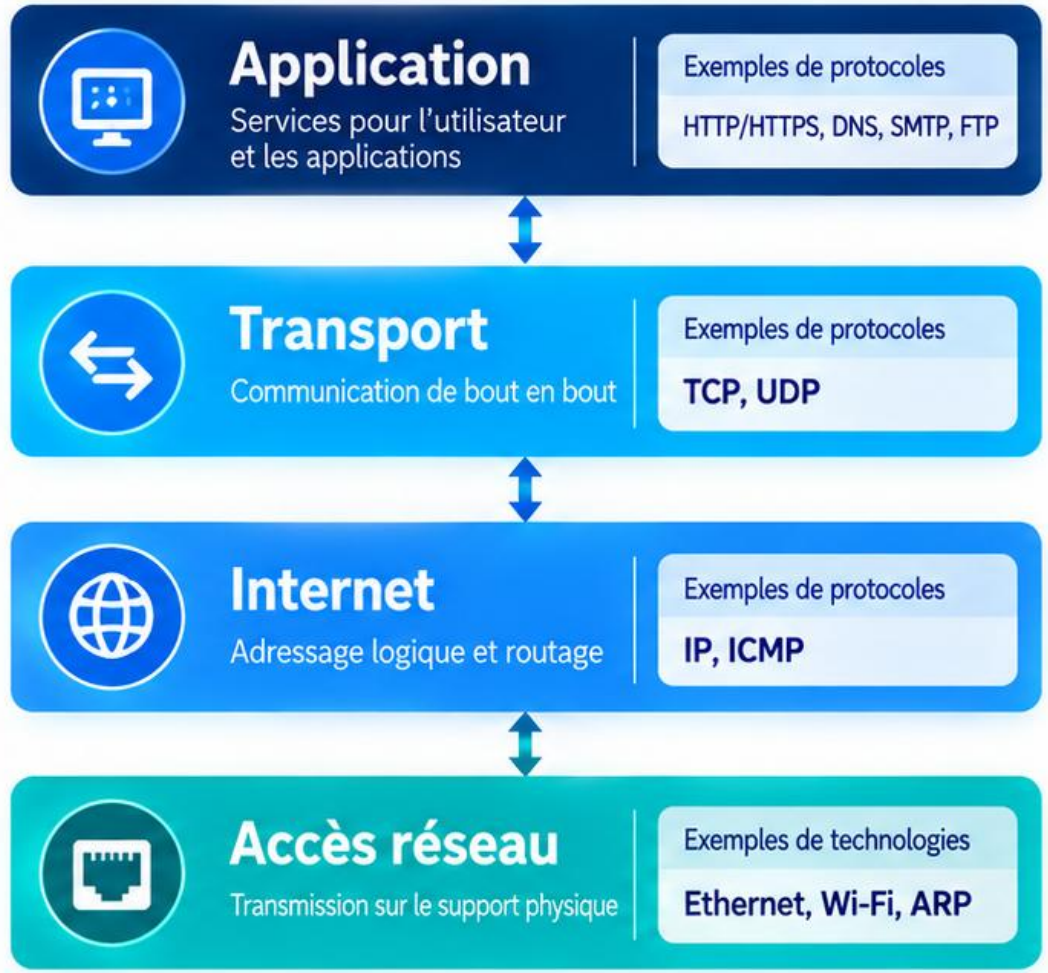
Rôle de chaque couche

Application :
fournit les services réseau aux applications.

Transport :
segmente, transporte et fiabilise les échanges.

Internet :
permet l'acheminement des paquets entre réseaux.

Accès réseau :
envoie les données sur le média physique.



Exemple d'encapsulation

Données	Données de l'application (ex. : page web, e-mail, fichier)
Segment	Ajout de l'en-tête transport (TCP ou UDP)
Paquet	Ajout de l'en-tête IP (adressage et routage)
Trame	Ajout de l'en-tête et de la bande de fin (Ethernet, Wi-Fi)

Chaque couche ajoute ses propres informations de contrôle.

Idée clé : le modèle TCP/IP organise la communication réseau en couches,
pour simplifier la conception, l'interopérabilité et le dépannage.

Une petite histoire…

Nous sommes en 2008, dans l'équipe réseau d'une grande entreprise.

20 routeurs

Un réseau déjà conséquent à administrer.

40 switches

VLAN, trunks, STP, accès utilisateurs…

Plusieurs sites

Des configurations différentes selon les lieux.

Quelques datacenters

Des flux critiques à maintenir disponibles.

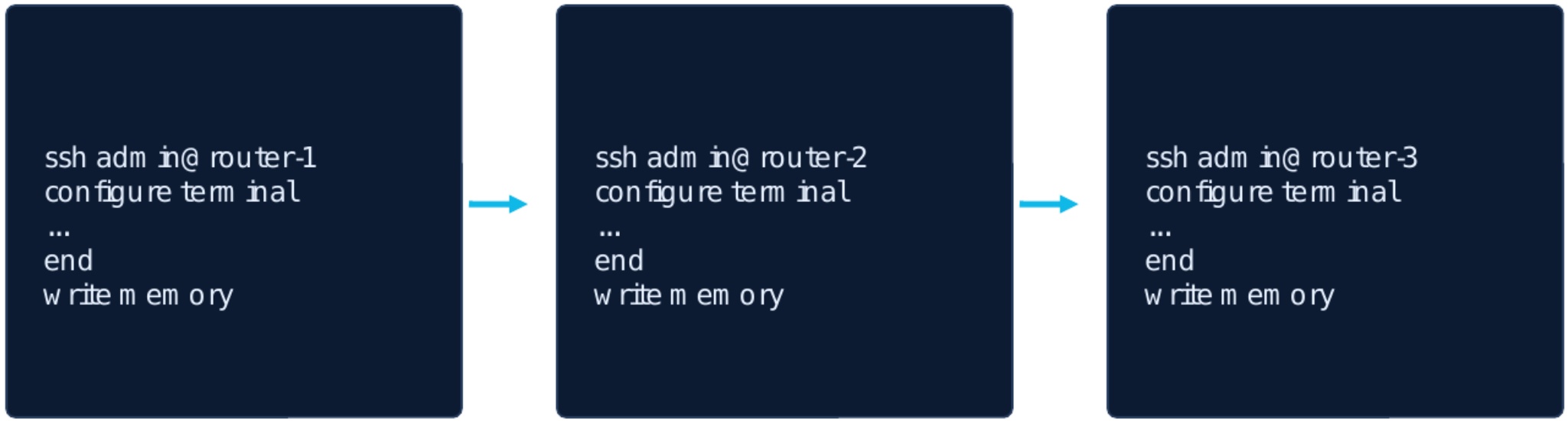
Lundi matin, votre responsable arrive :

« Nous devons modifier une politique réseau sur tous les équipements avant midi. »

LE PROBLÈME COMMENCE ICI

À ce moment-là, pas de magie.

On ouvre un terminal. Puis un autre. Puis encore un autre.



Router-1 **Router-2** **Router-3**

Puis le 4e… le 5e… le 20e…

Et si vous en aviez 2 000 ?

La question-pivot

À laquelle nous allons répondre plusieurs fois pendant le cours.

> **Peut-on continuer à gérer les réseaux de cette manière quand les infrastructures deviennent gigantesques ?**

Temps

Chaque changement doit être répété.

Erreurs

Plus on répète, plus on augmente le risque.

Visibilité

Chaque équipement ne voit qu'une partie du réseau.

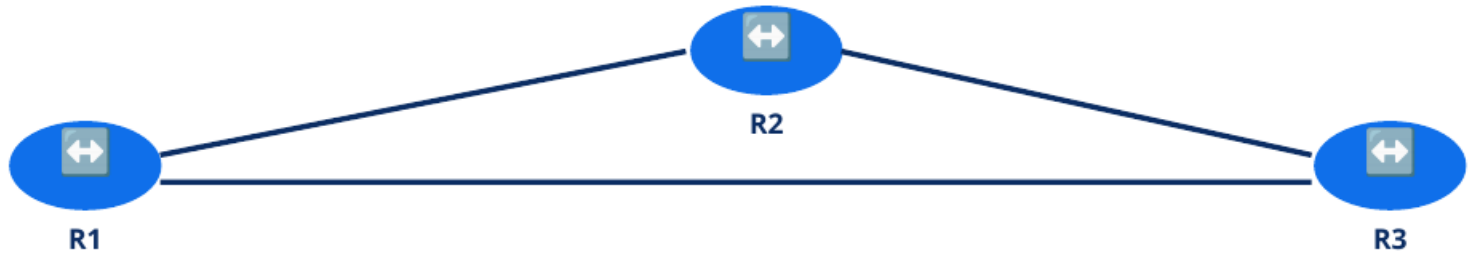
Cohérence

Maintenir une politique identique partout devient difficile.

C'est exactement là que commence l'histoire du SDN.

La philosophie du réseau classique

Chaque équipement est intelligent et prend ses propres décisions.



Plan de contrôle Plan de données	Plan de contrôle Plan de données	Plan de contrôle Plan de données

Le routeur connaît ses routes

Il échange des informations de routage et calcule localement son meilleur chemin.

Le switch connaît ses règles

Il applique localement ses décisions de commutation et de filtrage.

Intelligence distribuée

Chaque équipement possède son propre plan de contrôle.

Architecture réseau et routage : comment les paquets trouvent leur chemin ?

Le routage permet d'acheminer les paquets entre différents réseaux en choisissant le meilleur chemin.



Table de routage de R1

Réseau de destination	Masque	Prochain saut	Interface
192.168.1.0	255.255.255.0	-	LAN (192.168.1.1)
10.0.0.0	255.255.255.252	-	WAN (10.0.0.1)
192.168.2.0	255.255.255.0	10.0.0.2	WAN (10.0.0.1)
0.0.0.0	0.0.0.0	10.0.0.2	WAN (10.0.0.1)

Table de routage de R2

Réseau de destination	Masque	Prochain saut	Interface
192.168.2.0	255.255.255.0	-	LAN (192.168.2.1)
10.0.0.0	255.255.255.252	-	WAN (10.0.0.2)
192.168.1.0	255.255.255.0	10.0.0.1	WAN (10.0.0.2)
0.0.0.0	0.0.0.0	(vers Internet)	WAN (vers FAI)

Chemin d'un paquet de PC1 vers PC2 (192.168.1.10 → 192.168.2.10)



Idées clés

- Chaque routeur prend une décision en fonction de sa table de routage.
- Le prochain saut est l'adresse du routeur suivant.
- Le routage permet d'interconnecter plusieurs réseaux.
- Une route par défaut permet d'atteindre des réseaux inconnus (ex. Internet).

Protocoles de routage (exemples)

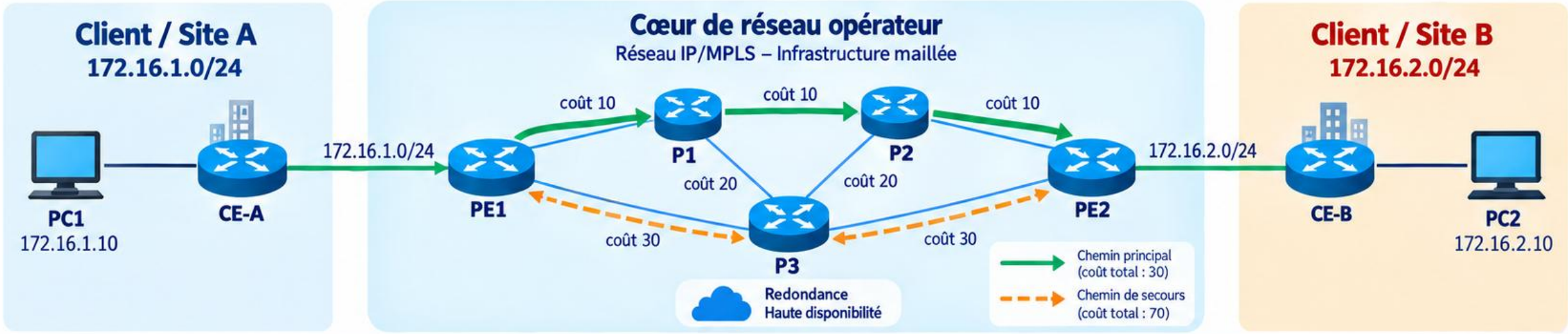
- **Routage statique** : configuration manuelle.
- **RIP** : simple, basé sur le nombre de sauts.
- **OSPF** : plus performant, utilisé en entreprise.
- **BGP** : utilisé sur Internet entre grands opérateurs.

À retenir

"Le routage, c'est permettre à chaque paquet de trouver le meilleur chemin vers sa destination, même si les réseaux sont différents."

Architecture maillée d'un opérateur : comment le routage choisit le meilleur chemin ?

Dans une infrastructure d'opérateur, plusieurs routeurs interconnectés offrent plusieurs chemins possibles pour acheminer les paquets de manière fiable et efficace.



Chemin d'un paquet de Site A vers Site B

1 Le site A envoie le paquet vers son routeur CE / PE.



2 Le réseau opérateur calcule le meilleur chemin selon la métrique / l'état des liens.



Les routeurs échangent leurs informations (OSPF / IS-IS) et calculent le meilleur chemin.

3 Le paquet traverse le cœur maillé via le chemin principal.



4 En cas de panne, un chemin de secours est utilisé automatiquement.



Pourquoi une architecture maillée ?

- **Plusieurs chemins disponibles**
  Plus de choix pour acheminer les paquets.
- **Tolérance aux pannes**
  Le trafic est automatiquement redirigé en cas d'incident.
- **Meilleure performance**
  Choix du chemin le plus rapide et le moins chargé.
- **Scalabilité pour de nombreux clients**
  Une infrastructure adaptée à la croissance du réseau.

Protocoles / technologies utilisés

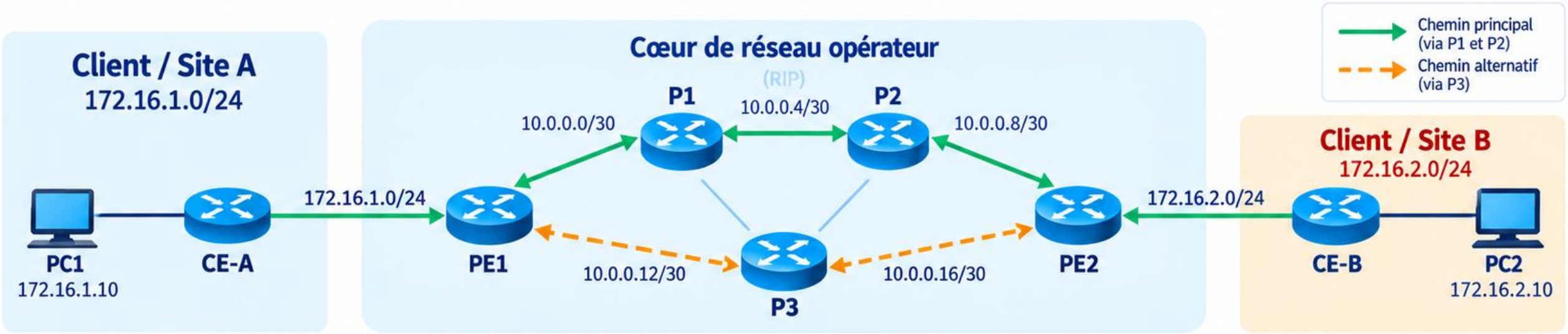
- **OSPF / IS-IS : calcul du meilleur chemin dans le cœur**
  Protocoles de routage interne (IGP) utilisés par les opérateurs pour connaître l'état du réseau et calculer les meilleurs chemins.
- **BGP : échange de routes entre domaines / clients**
  Utilisé pour échanger les routes avec d'autres opérateurs et pour gérer les routes des clients (VPN, Internet, etc.).
- **MPLS : transport efficace dans le réseau opérateur**
  Permet de transporter le trafic des clients avec des labels et d'assurer des services de bout en bout (VPN, QoS, etc.).

À retenir

À retenir : dans un réseau maillé d'opérateur, le routage ne suit pas un seul chemin fixe ; il choisit le meilleur chemin disponible et bascule en cas d'incident.

Même architecture avec RIP (Cisco)

Configuration distribuée : chaque routeur apprend les routes via **RIP**



CE-A (Cisco IOS)

```
router rip
 version 2
 no auto-summary
 network 172.16.1.0
 network 10.0.0.0
```

PE1 (Cisco IOS)

```
router rip
 version 2
 no auto-summary
 network 10.0.0.0
```

P1 (Cisco IOS)

```
router rip
 version 2
 no auto-summary
 network 10.0.0.0
```

P2 / P3 (Cisco IOS)

```
router rip
 version 2
 no auto-summary
 network 10.0.0.0
```

(même configuration pour P2 et pour P3)

PE2 (Cisco IOS)

```
router rip
 version 2
 no auto-summary
 network 10.0.0.0
```

CE-B (Cisco IOS)

```
router rip
 version 2
 no auto-summary
 network 172.16.2.0
 network 10.0.0.0
```

Ce que fait RIP

- Échange périodique des routes (toutes les 30 secondes)
- Métrique = nombre de sauts (*hop count*)
- Chaque routeur décide localement de son meilleur chemin

Philosophie classique :

intelligence distribuée, configuration routeur par routeur.



Simple, robuste, pédagogique !

Important : cette architecture fonctionne.

Le SDN ne rend pas soudainement le réseau classique “mauvais”.

Elle fonctionne très bien.

Et elle fonctionne encore aujourd’hui.

Mais… **plus le réseau grandit, plus sa gestion devient compliquée.**



La difficulté n’est pas seulement technique : elle devient opérationnelle.

Le constat qui dérange

Le problème n’est pas forcément là où on l’imagine.

Ce n’est pas TCP/IP.

Les paquets peuvent très bien continuer à circuler avec les mêmes protocoles.

Ce n’est pas le routage.

OSPF, BGP, RIP… savent déjà construire et utiliser des chemins.

Ce n’est pas “le câble”.

Le problème apparaît surtout dans la façon de gérer l’ensemble.

> **Le vrai problème devient : comment piloter des milliers d’équipements de manière cohérente ?**

La douleur n’est plus seulement dans le forwarding. Elle est dans le pilotage.

Et Google se pose une autre question…

C’est ici que l’intuition commence à se retourner.

> **Pourquoi laisser chaque équipement décider seul, alors que nous pourrions avoir une vision globale du réseau ?**

Vision locale

Chaque routeur connaît sa partie du réseau et prend sa décision localement.

Vision globale

Un système central peut observer l’ensemble et optimiser les décisions selon un objectif commun.

C’est ce changement de perspective qui ouvre la porte au SDN.

La bascule : « Et si… ? »

On ne pense plus d’abord aux équipements. On pense à l’intention.

Avant

« Je configure mes routeurs un par un. »

→ connexion locale
→ commandes spécifiques
→ répétition
→ vérification équipement par équipement



Après

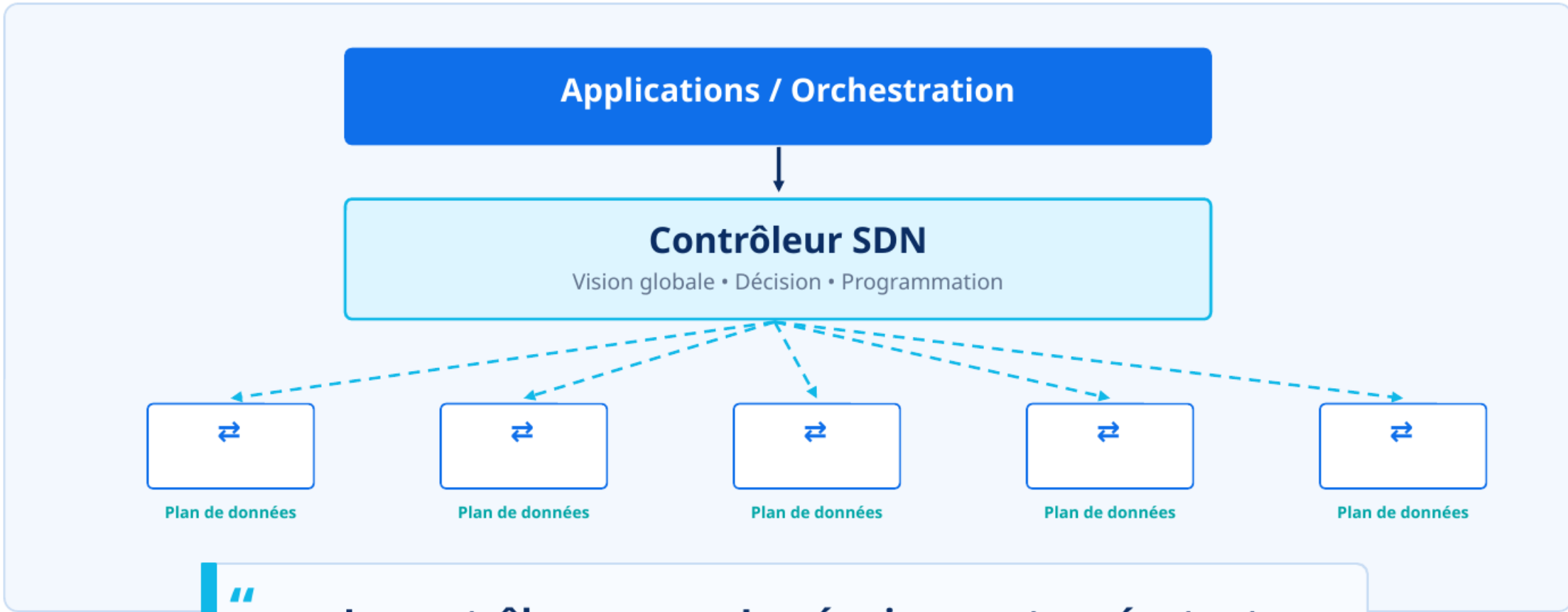
« Je définis ce que je veux que le réseau fasse. »

→ politique globale
→ automatisation
→ contrôleur
→ déploiement cohérent

Un logiciel se charge ensuite de programmer le réseau.

La révélation : Software-Defined Networking

Le SDN change surtout la façon de piloter le réseau.



> **Le contrôleur pense. Les équipements exécutent.**

Séparer décision et exécution

Une distinction simple, mais fondamentale.

Plan de contrôle

Il décide.

- vue du réseau
- choix des chemins
- politiques
- réactions aux événements
- orchestration

Plan de données

Il exécute.

- transfert des paquets
- application des règles
- compteurs
- actions sur les flux

Le réseau devient programmable sans cesser d'être un réseau IP.

Historique du SDN (Software-Defined Networking)

Le **SDN** est une nouvelle approche des réseaux qui sépare le **plan de contrôle** (la prise de décision) du **plan de données** (le transfert des paquets) pour rendre les réseaux plus flexibles, programmables et faciles à gérer.

Avant 2000

Réseaux traditionnels (longtemps dominants)

Les équipements réseau (routeurs, commutateurs) combinent dans un méme appareil le plan de contrôle et le plan de données.



Plan de contrôle (routage, décisions)

Plan de données (transfert de paquets)

Peu flexibles, configuration manuelle, difficiles à gérer à grande échelle.

2001 – 2006

Premières idées de séparation

Des chercheurs commencent à explorer l'idée de séparer le plan de contrôle et le plan de données pour plus de flexibilité.

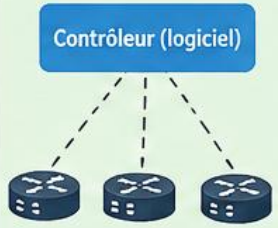


Premiers travaux académiques sur une architecture réseau plus modulaire.

2006 – 2008

Programmabilité des réseaux

Des projets comme Ethane (Stanford, 2006) montrent qu'un contrôleur centralisé peut gérer un réseau de manière logicielle.

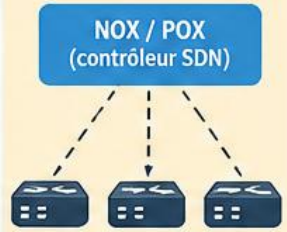


Preuve de concept : le réseau peut être contrôlé par un logiciel central.

2008 – 2011

Émergence du terme SDN

Le terme Software-Defined Networking (SDN) se généralise. Le projet NOX (puis POX) propose un contrôleur open source pour expérimenter le SDN.

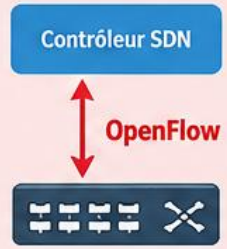


Le SDN devient un concept reconnu dans la communauté networking.

2011 – 2013

Standardisation avec OpenFlow

OpenFlow (2011) devient le premier standard permettant la communication entre le contrôleur et les équipements réseau (switches compatibles).

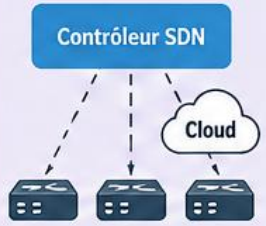


OpenFlow facilite l'adoption du SDN dans les laboratoires et les entreprises.

2013 – 2016

Adoption et maturation

De grandes entreprises (IBM, Cisco, HP, etc.) et des initiatives comme l'ONF (Open Networking Foundation) soutiennent le SDN. Le SDN est utilisé dans les data centers et les réseaux cloud.

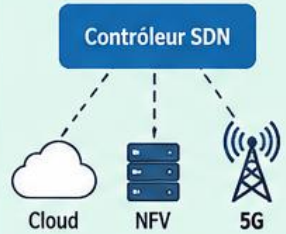


Le SDN s'intègre dans des environnements réels (data centers, cloud, opérateurs).

2016 – Aujourd'hui

Évolution et nouvelles tendances

Le SDN évolue vers des réseaux plus intelligents avec l'intégration de l'IA, du NFV (Network Functions Virtualization) et des réseaux 5G. Il est largement adopté dans les infrastructures cloud et les réseaux d'entreprise modernes.



Le SDN devient un élément clé des réseaux modernes, plus automatisés, flexibles et intelligents.

Qu'est-ce que le SDN ?

Le Software-Defined Networking (SDN) est une architecture **réseau qui sépare :**

- **le plan de contrôle** : qui décide où envoyer les paquets (géré par un contrôleur logiciel),
- **le plan de données** : qui se charge uniquement du transfert des paquets (switches/routeurs simples).

Cette séparation permet de programmer et de gérer le réseau plus facilement, via des interfaces logicielles (comme OpenFlow).

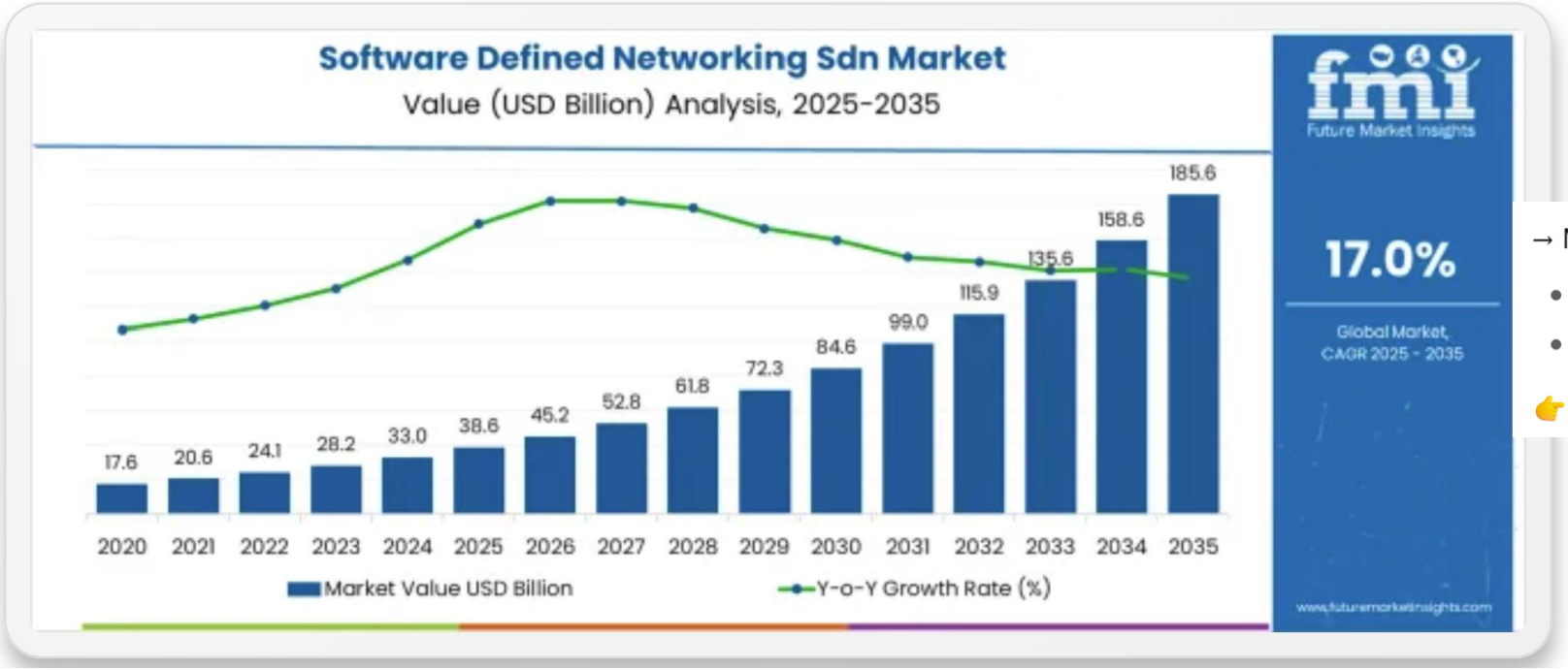
Pourquoi le SDN est important ?

- **Plus de flexibilité :** le réseau peut être reconfiguré rapidement par logiciel.
- **Gestion centralisée :** une vue globale du réseau.
- **Réduction des coûts :** utilisation d'équipements plus simples.
- **Innovation :** facilite l'automatisation, le cloud et les nouvelles applications (IoT, 5G, etc.).

En résumé

Le SDN est le résultat de plus de 20 ans de recherches et d'innovations. Parti d'une idée académique, il est devenu une technologie clé des réseaux modernes, grâce à la standardisation (OpenFlow) et à l'adoption par l'industrie. Aujourd'hui, il constitue la base des réseaux intelligents et programmables, en combinaison avec le cloud, la virtualisation (NFV) et l'IA.

Analyse du marché mondial du Software Defined Networking (SDN) réalisée par *Future Market Insights (FMI)*, couvrant la période **2025–2035**



→ Montre la **valeur du marché SDN** en milliards de dollars US.

- En 2020 : **17,6 milliards USD**
- En 2035 : **185,6 milliards USD**

👉 Cela représente une **croissance de plus de 10 fois** en 15 ans.

Implications économiques :

Le SDN devient une **technologie clé pour la 5G, les Data Centers et le Cloud**.

Il est essentiel dans la **virtualisation du réseau (NFV)** et la **cybersécurité adaptative**.

Les **grands acteurs comme Google, Microsoft, Amazon, Huawei et Cisco** investissent massivement dans cette technologie pour automatiser et sécuriser leurs infrastructures.

Software-Defined Networking



















Un écosystème devenu très large

Le SDN dépasse largement Google et OpenFlow.

Google

Microsoft

Meta

Amazon / AWS

Opérateurs télécoms

Cisco

Juniper

Nokia

ONOS

Contrôleur SDN open source historiquement très présent dans les environnements opérateurs.

OpenDaylight

Plateforme open source de contrôleur et d'automatisation réseau.

Le vrai changement : une nouvelle philosophie

Le SDN n’est pas simplement “un nouveau protocole réseau”.

Réseau traditionnel

Question dominante :

« Quel équipement dois-je configurer ? »

→ vision équipement
→ commandes locales
→ intelligence distribuée

Philosophie SDN

Question dominante :

« Quelle politique veux-je appliquer au réseau ? »

→ intention
→ automatisation
→ vision globale

> **On passe d’un réseau que l’on configure équipement par équipement à un réseau que l’on pilote comme un système logiciel.**

L’architecture traditionnelle

Qu'est-ce qu'un plan de contrôle ?

Le plan de contrôle est un composant crucial du réseau, qui prend des décisions sur la manière dont les données doivent être gérées, acheminées et traitées.

Il agit comme un superviseur de données, coordonnant la communication entre les différents composants et collectant les données du plan de données.

Les plans de contrôle utilisent divers protocoles, tels que :

- Protocoles de routage (tels que BGP, OSPF et IS-IS)
- Protocoles de gestion de réseau (SNMP)
- Protocoles de la couche application (HTTP et FTP)

Qu'est-ce qu'un plan de données ?

Le plan de données est responsable du mouvement réel des données d'un système à l'autre. C'est le cheval de bataille qui fournit les données aux utilisateurs finaux à partir des systèmes et vice versa.

Voici quelques exemples de plans de données :

- Réseaux Ethernet
- Réseaux Wi-Fi
- Réseaux cellulaires
- Communications par satellite

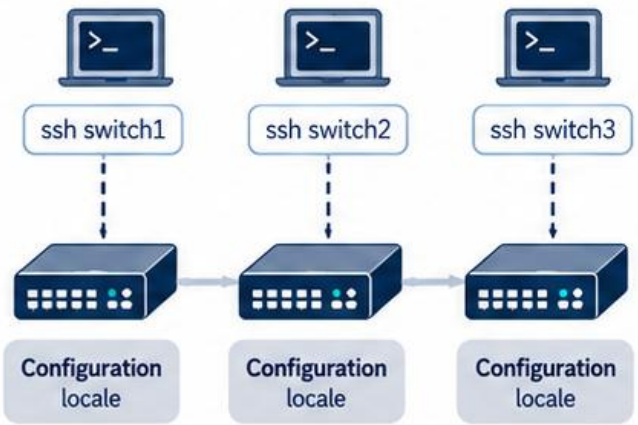
Pourquoi gère-t-on encore le réseau équipement par équipement ?

Et si le vrai problème n'était pas le trafic… mais la façon de piloter le réseau ?

- Faut-il se connecter à chaque switch pour appliquer une simple politique ?
- Peut-on automatiser un grand réseau avec des commandes locales répétées ?
- Pourquoi le réseau ne réagit-il pas comme un système intelligent et global ?

Philosophie classique

Un réseau de boîtes à configurer



- **Intelligence répartie** dans chaque équipement
- **Configuration locale / manuelle**
- **Répétition des commandes**
- **Visibilité limitée**

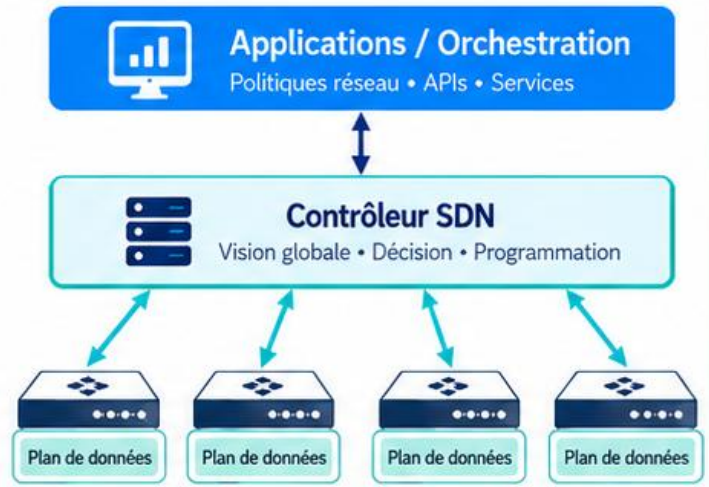
⚠ **Même règle** = plusieurs connexions, plusieurs commandes, plus de risques d'erreur.

La bascule

Changement de philosophie

Philosophie SDN

Un réseau piloté de manière centralisée et intelligente



- **Intelligence centralisée**
- **Politique globale**
- **Programmation / automatisation**
- **Vision réseau unifiée**

✓ **Une seule intention = politique déployée partout.**

Le SDN ne change pas seulement les outils : il change la philosophie du réseau.

On passe d'un réseau configuré boîte par boîte à un réseau piloté comme un logiciel.

Introduction au SDN

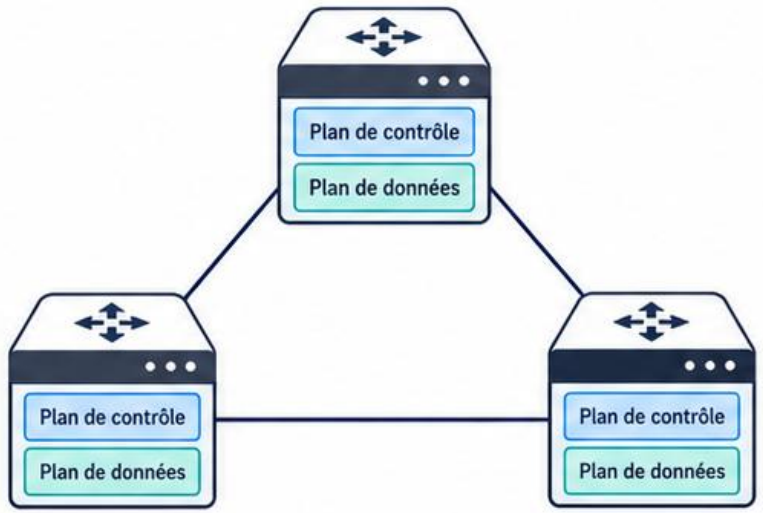
Software-Defined Networking : une approche où le contrôle du réseau est séparé des équipements de transfert pour rendre le réseau programmable, centralisé et flexible.

C'est quoi le SDN ?

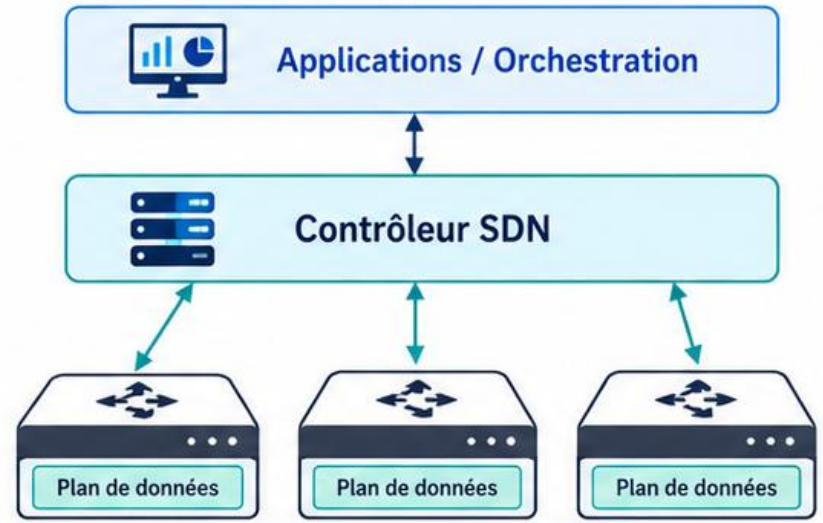
- Séparation du plan de contrôle et du plan de données
- Contrôleur centralisé
- Configuration et automatisation par logiciel

Architecture classique

- Configuration équipement par équipement
- Gestion plus rigide
- Automatisation limitée



Architecture SDN



- Contrôle centralisé
- Programmable
- Déploiement rapide des politiques
- Vision globale du réseau

	Architecture classique	Architecture SDN
Intelligence du réseau	Distribuée dans les équipements	Centralisée dans le contrôleur
Configuration	Manuelle / locale	Programmable / automatisée
Flexibilité	Faible	Élevée
Visibilité	Limitée	Globale

Les avantages du SDN

Pourquoi le Software-Defined Networking transforme les réseaux modernes

1 Gestion centralisée



Une vue globale du réseau et une administration simplifiée.

3 Automatisation



Déploiement plus rapide des politiques réseau et réduction des tâches manuelles.

5 Sécurité renforcée



Contrôle fin des flux, segmentation et réaction plus rapide aux incidents.

7 Extensibilité

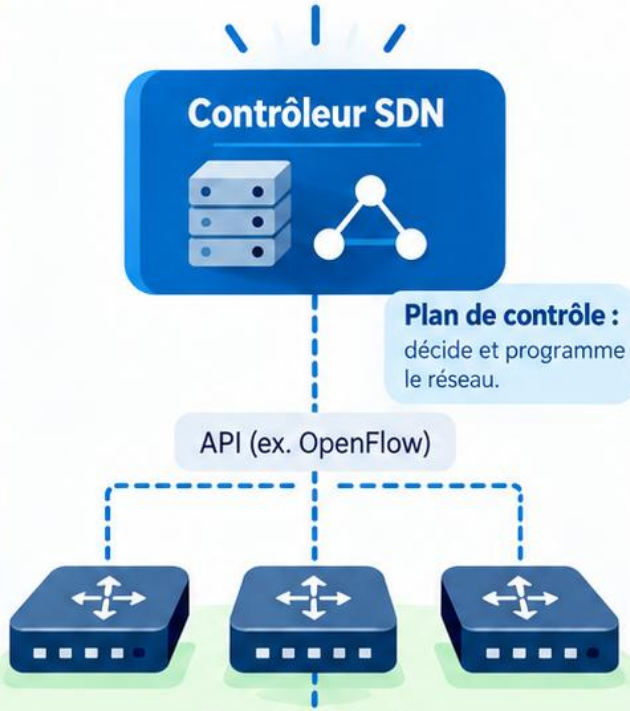


Le réseau évolue plus facilement pour absorber la croissance.

9 Contrôle du trafic



Meilleure visibilité et pilotage des flux réseau.



2 Flexibilité



Reconfiguration rapide selon les besoins métiers et techniques.

4 Réduction des coûts



Meilleure utilisation des équipements et baisse des coûts d'exploitation.

6 Innovation et agilité



Intégration facilitée avec le cloud, la virtualisation, l'IoT et la 5G.

8 Réduction de la surcharge



Moins de configuration manuelle et moins de complexité opérationnelle.

11 Gestion multi-cloud simplifiée



Pilotage plus cohérent des environnements hybrides et multi-cloud.

10 Réduction des interruptions



Diminution des temps d'arrêt grâce à une gestion plus réactive.

En résumé :

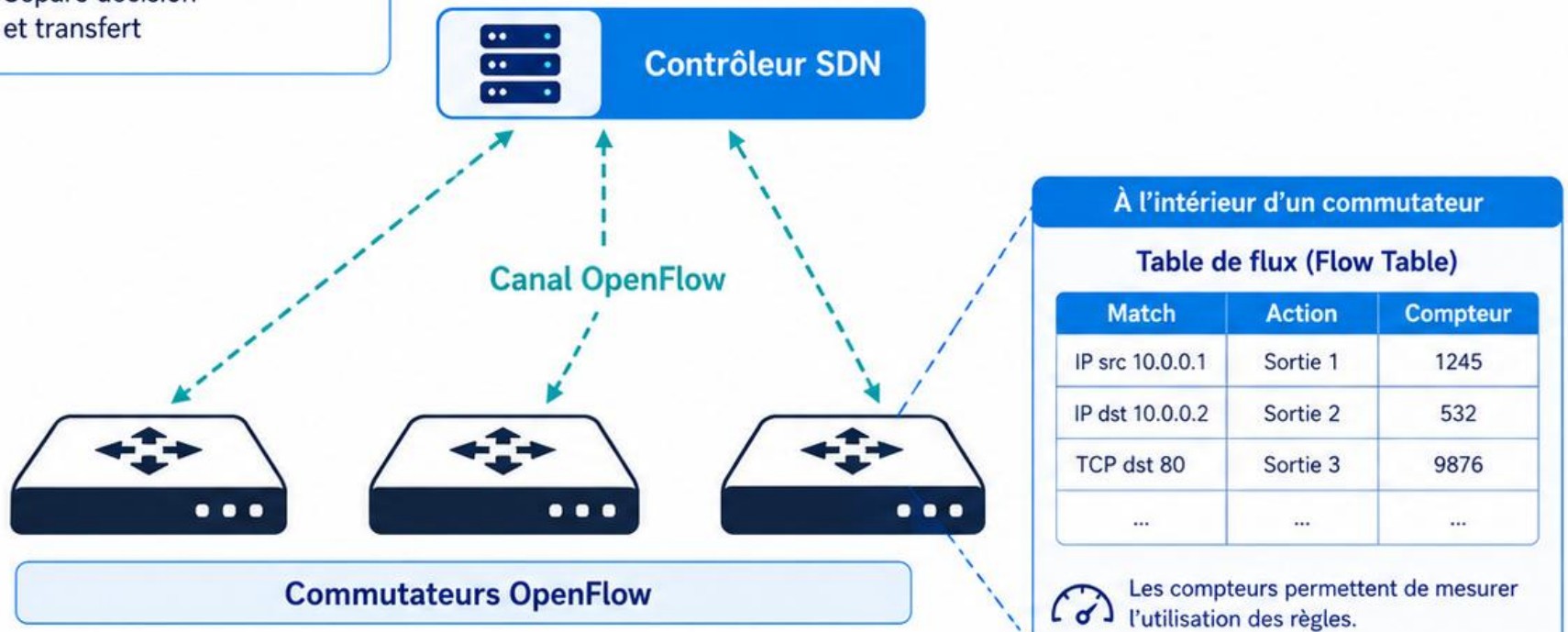
le SDN rend les réseaux **plus intelligents, programmables, évolutifs et faciles à gérer.**

Introduction à OpenFlow

OpenFlow est un protocole southbound qui permet au contrôleur SDN de communiquer avec les commutateurs pour installer et gérer les règles de transfert.

C'est quoi OpenFlow ?

- Protocole de communication SDN
- Permet de programmer les règles de transfert
- Sépare décision et transfert



À l'intérieur d'un commutateur

Table de flux (Flow Table)

Match	Action	Compteur
IP src 10.0.0.1	Sortie 1	1245
IP dst 10.0.0.2	Sortie 2	532
TCP dst 80	Sortie 3	9876
...	...	...

Les compteurs permettent de mesurer l'utilisation des règles.

Comment ça marche ?

1. **Un paquet inconnu arrive**
   Le switch ne trouve pas de règle correspondante dans sa table de flux.
2. **Le switch envoie un Packet-In au contrôleur**
   Le paquet et ses métadonnées sont envoyés au contrôleur.
3. **Le contrôleur décide la règle**
   Le contrôleur calcule la meilleure action selon sa logique et sa vue réseau.
4. **Le switch reçoit un Flow-Mod et applique l'action**
   La règle est installée dans la table de flux et appliquée aux paquets suivants.

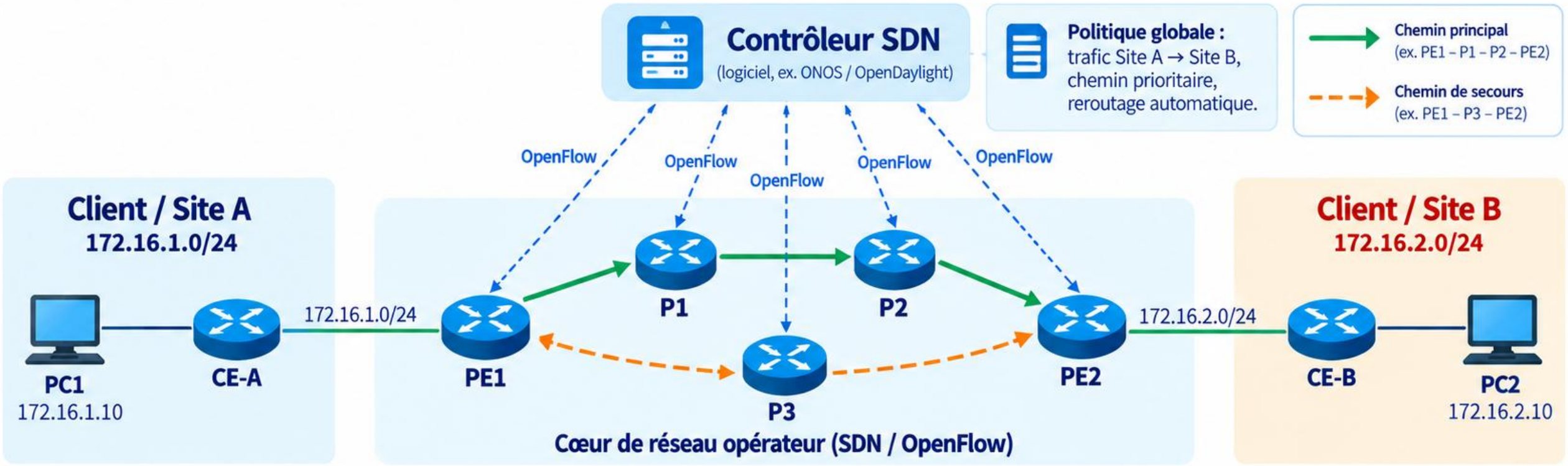
Rôle du contrôleur : intelligence
Prend les décisions globales, calcule les règles et pilote le réseau.

Rôle du switch : exécution
Applique les règles de transfert à haute performance.

Avantages : flexibilité, automatisation, visibilité
Programmation centralisée, déploiement rapide des services et meilleure visibilité du réseau.

Même architecture avec SDN / OpenFlow

Contrôle centralisé : le contrôleur pousse les règles vers les équipements



Exemple de règle OpenFlow

```
match: dst 172.16.2.0/24          # chemin principal
action: output vers P1

match: dst 172.16.2.0/24          # en cas de panne
action: output vers P3
```

Ce que change OpenFlow

1. **Décision centralisée**
   Le contrôleur calcule les meilleurs chemins.
2. **Règles déployées automatiquement**
   Les équipements reçoivent les règles via OpenFlow.
3. **Vision globale du réseau**
   Une vue d'ensemble pour une gestion plus simple et plus efficace.

Philosophie SDN

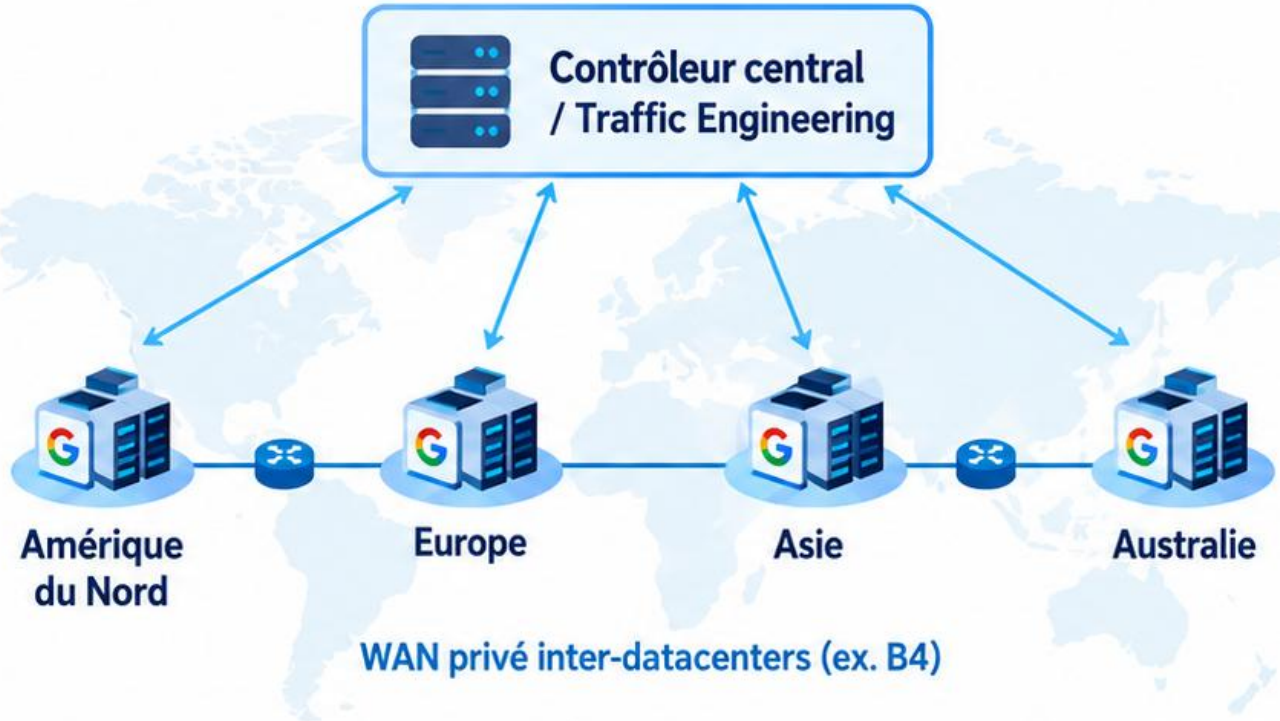
On configure l'intention une fois, le contrôleur applique partout.

Prenons Google... pourquoi le SDN ?

Quand le réseau devient immense, une gestion équipement par équipement ne suffit plus.

Le contexte chez Google

- Des datacenters répartis dans le monde
- Un trafic énorme : Search, YouTube, Gmail, Cloud
- Besoin d'optimiser les liens et les chemins



Pourquoi utiliser le SDN ?

- Vision globale du réseau
- Décisions centralisées
- Meilleure utilisation de la bande passante
- Réaction rapide aux variations de trafic

Idée clé

Au lieu de laisser chaque routeur décider seul, Google pilote le réseau avec une logique centralisée et programmable.

Exemples célèbres : **B4, Espresso, B2.**

Le SDN chez Google

Quelques déploiements célèbres : B4, Espresso et B2

Contrôle centralisé / Orchestration
Vision globale • Politiques • Optimisation

Amérique du Nord — Europe — Asie — Amérique du Sud — Australie

1 B4

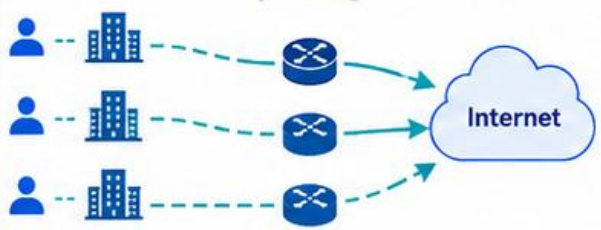
WAN inter-datacenters piloté par SDN



- traffic engineering centralisé
- meilleure utilisation des liens
- gestion du trafic entre datacenters

2 Espresso

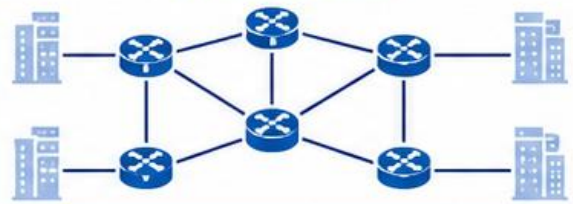
SDN pour la périphérie Internet / peering



- pilotage du trafic vers Internet
- choix dynamique des chemins
- meilleure qualité de service

3 B2

Infrastructure réseau Google à grande échelle



- automatisation
- contrôle centralisé
- haute disponibilité

Pourquoi c'est important ?

- réseau programmable
- décisions centralisées
- réaction rapide aux variations de trafic
- meilleure visibilité globale

Idée clé

Chez Google, le SDN ne sert pas seulement dans les datacenters :
il aide aussi à piloter le WAN, le peering et l'infrastructure réseau à très grande échelle.

L’architecture traditionnelle

Division en 3 plans : Application, Contrôle, Données



1️⃣ Plan de données (Data Plane)

👉 C’est le plan qui **transporte réellement les paquets** d’un point A à un point B.

- **OSI** :
  - Couche 1 (Physique)
  - Couche 2 (Liaison de données)
  - Couche 3 (Réseau, mais uniquement pour le forwarding des paquets)
- **TCP/IP** :
  - Accès réseau (Network Interface)
  - IP (forwarding pur)



🔹 Rôle : **acheminer les données** (bits → trames → paquets).

Exemple : une carte réseau transmet un paquet Ethernet vers le switch.

Division en 3 plans : Application, Contrôle, Données



2 Plan de contrôle (Control Plane)

👉 C'est le plan qui **prend les décisions** : comment les données doivent circuler.

- **OSI** :
  - Couche 3 (Réseau, partie routage)
  - Couche 4 (Transport, partie contrôle comme TCP handshake, congestion control)
- **TCP/IP** :
  - IP (routage, protocoles ICMP)
  - TCP/UDP (contrôle de flux, retransmission)



- Rôle : **décider du chemin**, gérer les états de connexion, adapter le flux.

Exemple : protocole OSPF calcule une table de routage, TCP ajuste la fenêtre de congestion.

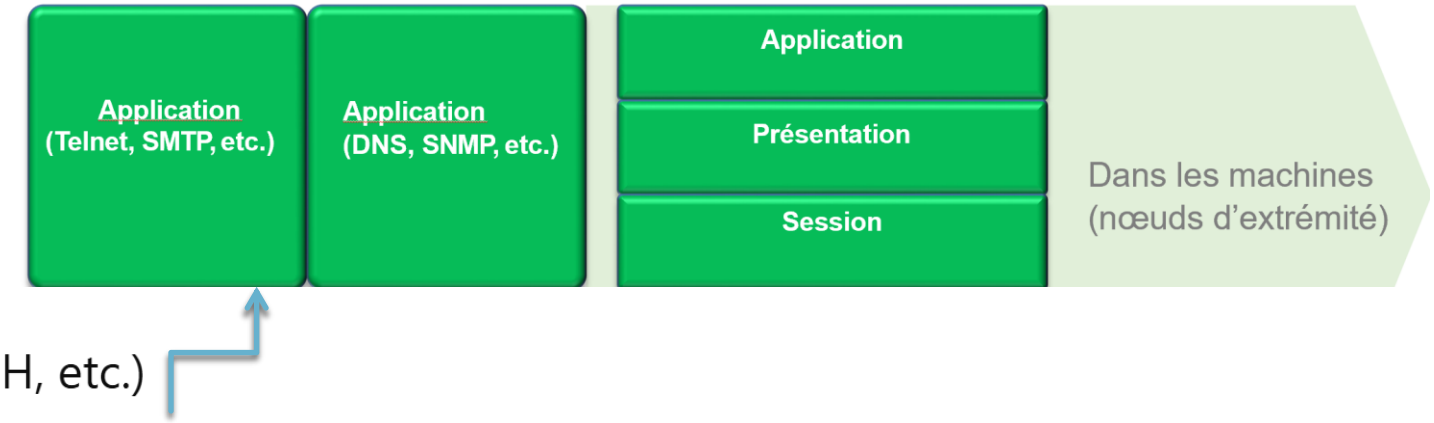
Division en 3 plans : Application, Contrôle, Données



3 Plan d'application (Application Plane)

👉 C'est le plan qui **utilise le réseau** pour fournir un service aux utilisateurs.

- **OSI** :
  - Couche 5 (Session)
  - Couche 6 (Présentation)
  - Couche 7 (Application)
- **TCP/IP** :
  - Application (HTTP, DNS, FTP, SSH, etc.)



🔹 Rôle : **offrir des services applicatifs** (web, mail, streaming, etc.).

Exemple : un navigateur envoie une requête HTTP vers un serveur web.

L’architecture traditionnelle



Qu'est-ce que le plan de contrôle ?

Le plan de contrôle est la partie d'un réseau qui contrôle la façon dont les paquets de données sont acheminés.

Le processus du routage, par exemple, est considéré comme faisant partie du plan de contrôle.

Les routeurs utilisent divers protocoles pour identifier les chemins du réseau, et ils stockent ces chemins dans des tables de routage.

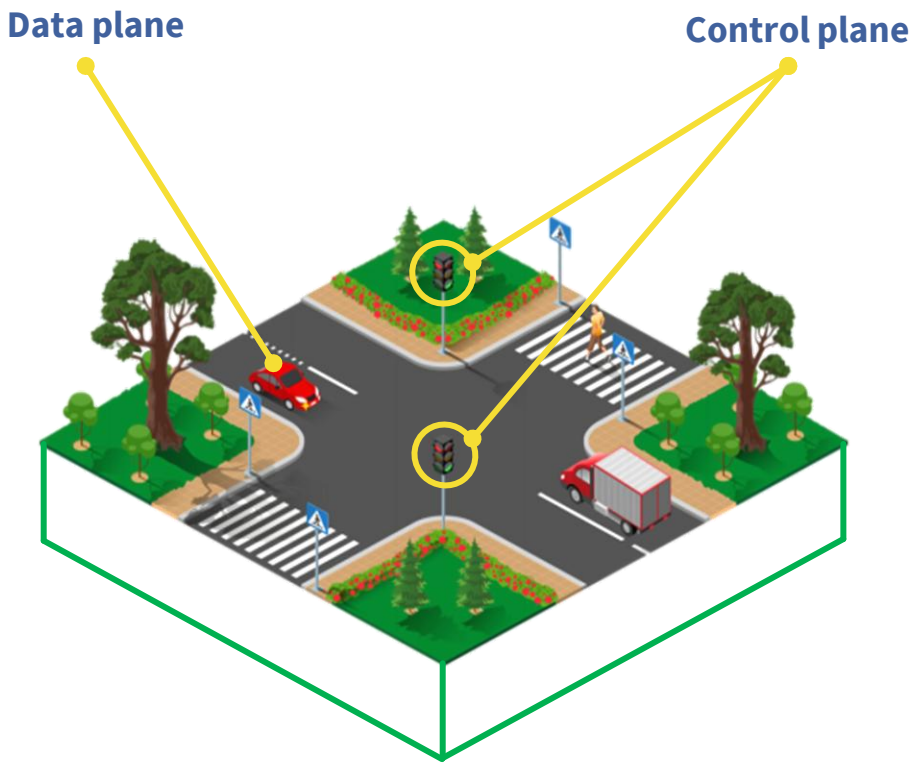
Qu'est-ce que le plan de données ? Qu'est-ce que le plan de transfert ?

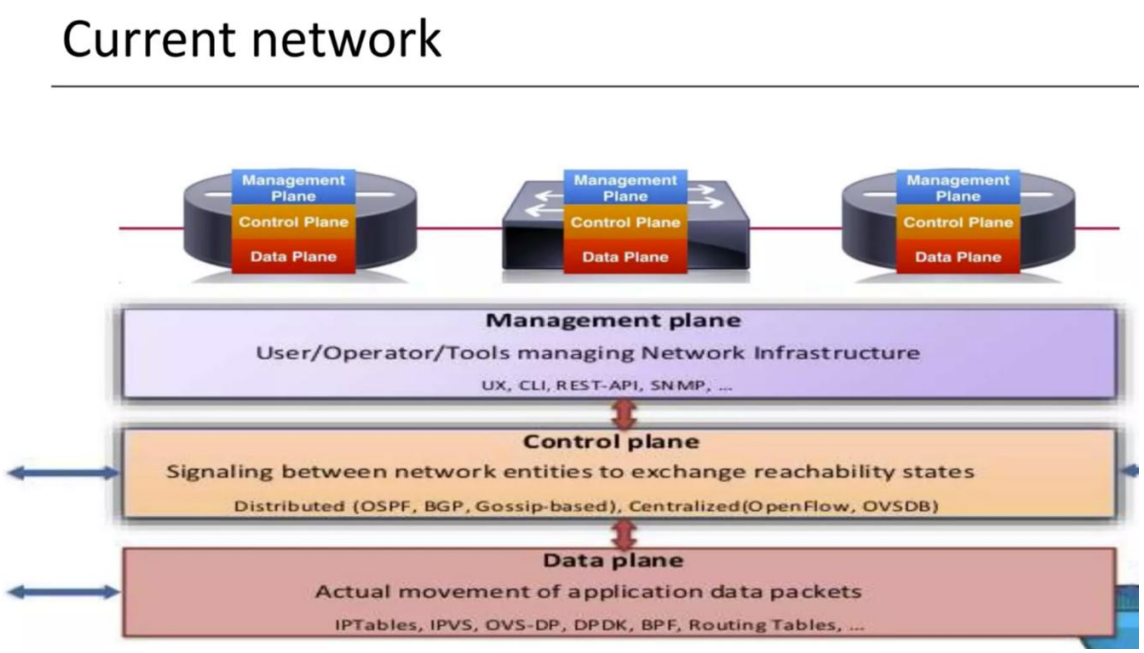
Contrairement au plan de contrôle, qui détermine comment les paquets doivent être transmis, le plan de données transmet réellement les paquets.

Pensez au plan de contrôle comme étant les feux de signalisation qui fonctionnent aux intersections d'une ville.

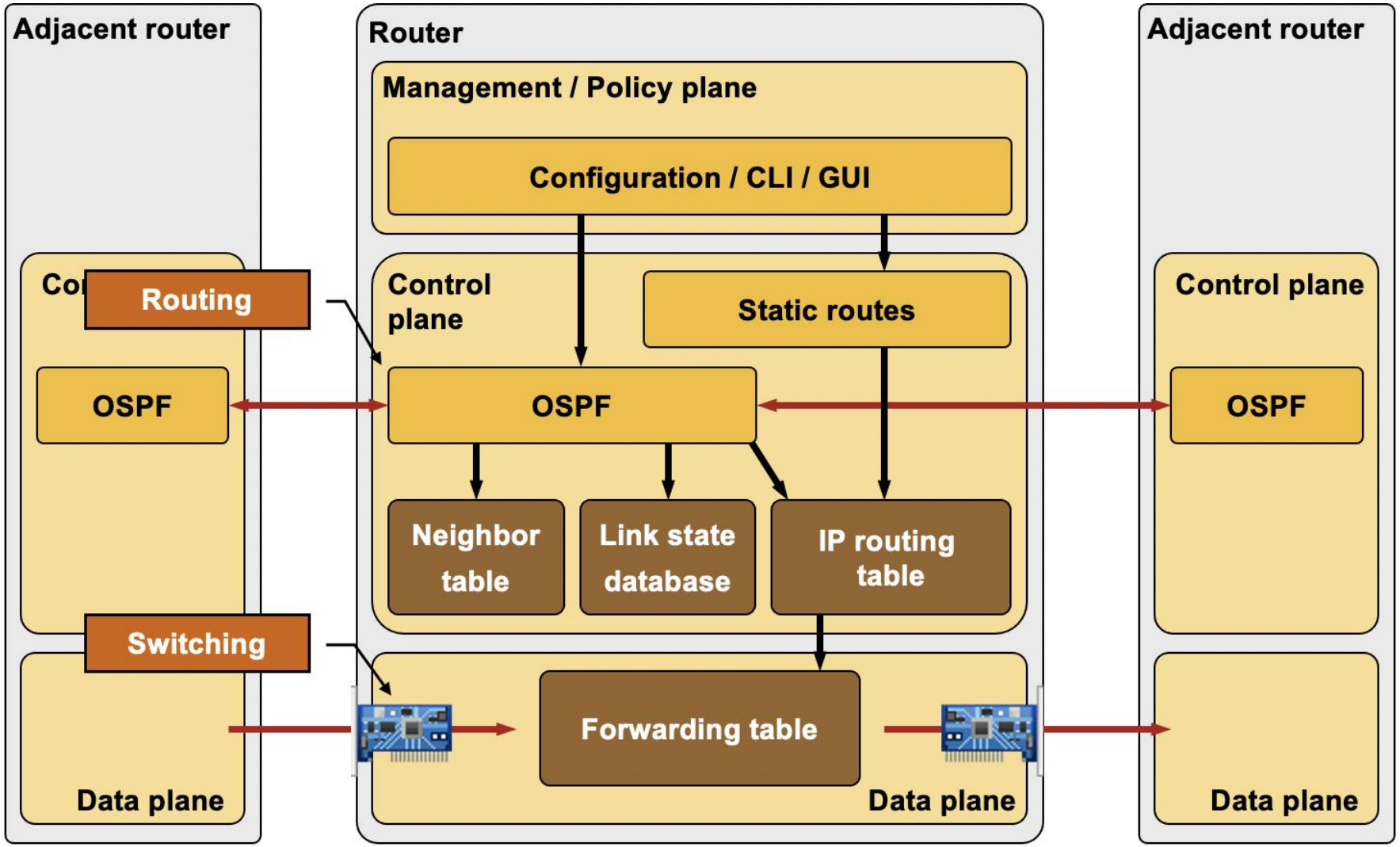
Le plan de données (ou plan d'acheminement), quant à lui, ressemble davantage aux voitures qui circulent sur les routes, s'arrêtent aux intersections et respectent les feux de signalisation.

L’architecture traditionnelle

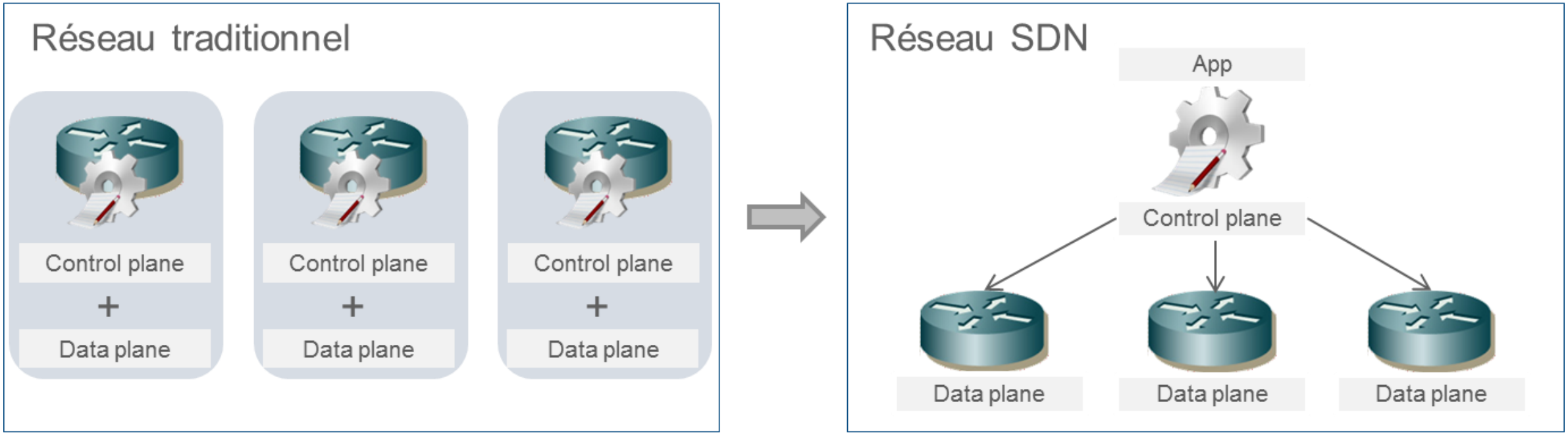




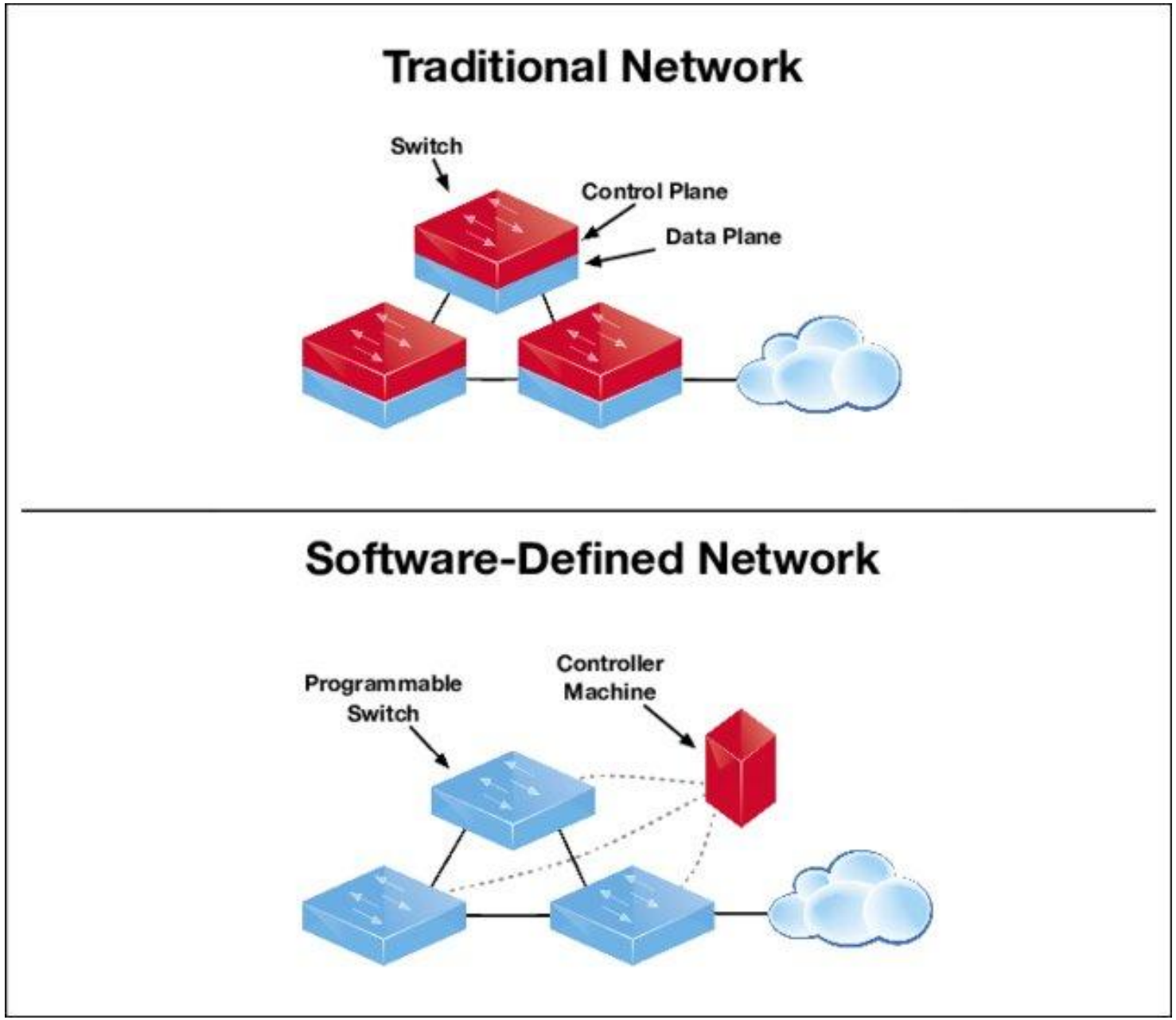
L’architecture traditionnelle



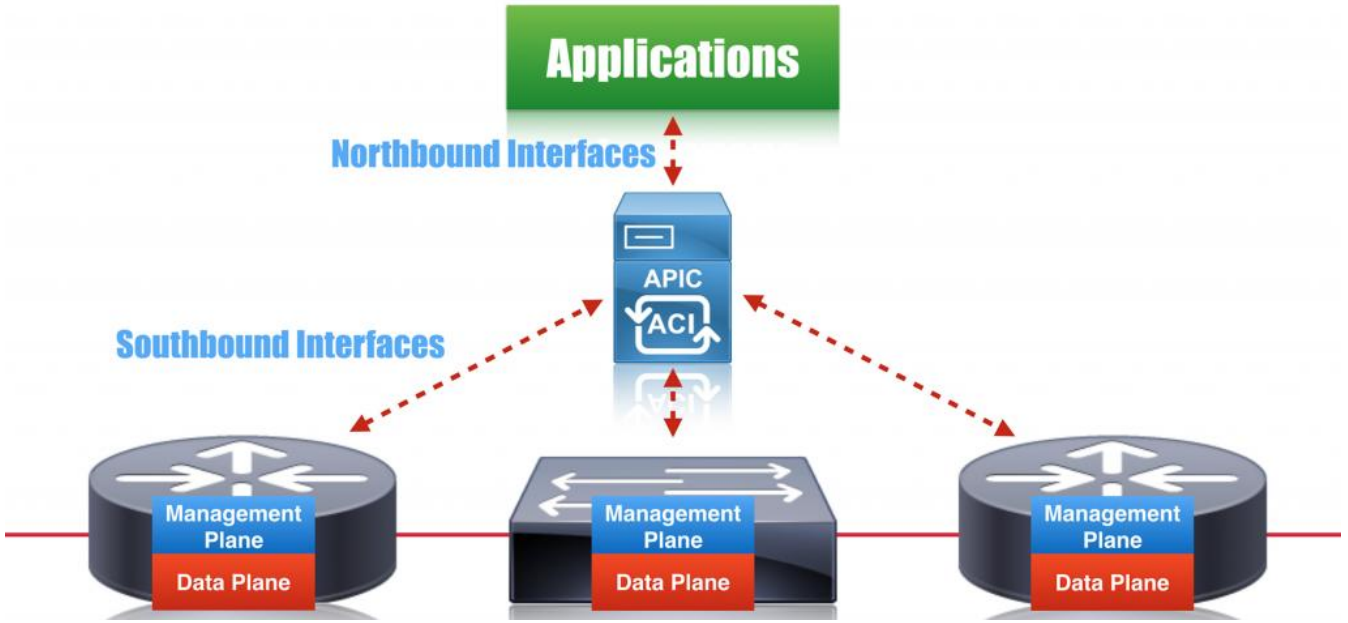
NFV, SDN : comment la virtualisation va modifier en profondeur les réseaux



L’architecture traditionnelle: Problématique et solution

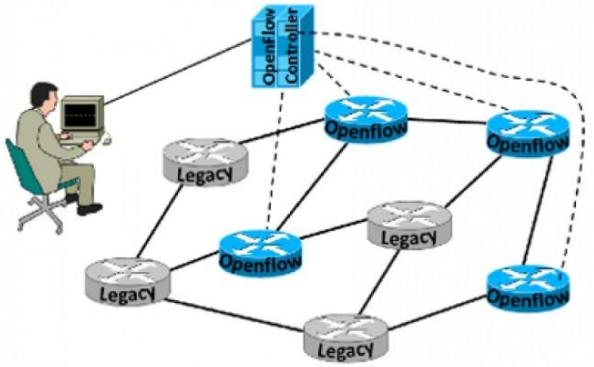


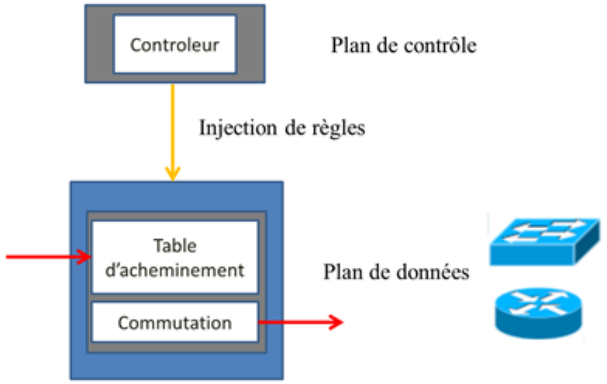
L’architecture traditionnelle: Problématique et solution

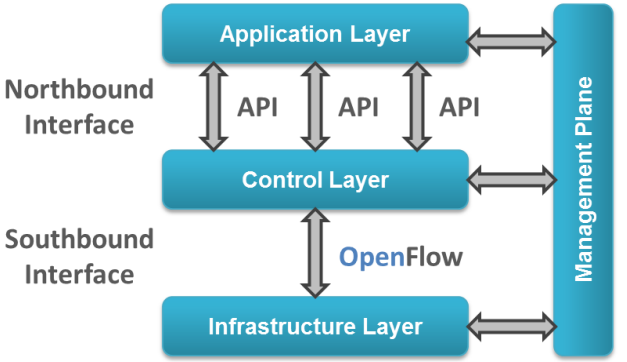


Qu'est-ce qu'un réseau défini par logiciel ?

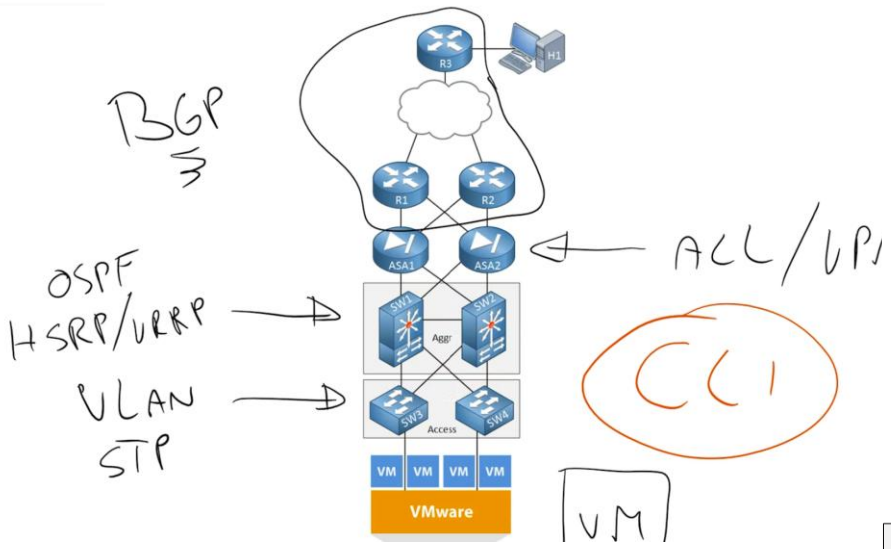
Software-defined networking (SDN) est une méthode de gestion et de configuration des réseaux par logiciel. La technologie SDN permet aux administrateurs informatiques de configurer leurs réseaux à l'aide d'une application logicielle, au lieu de modifier la configuration des équipements physiques. Le SDN est rendu possible par la séparation du plan de contrôle du plan d'acheminement/de données.



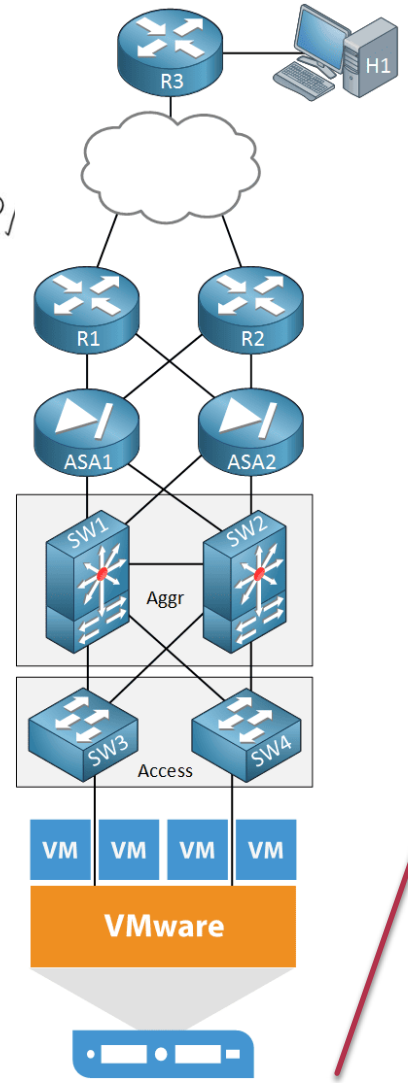




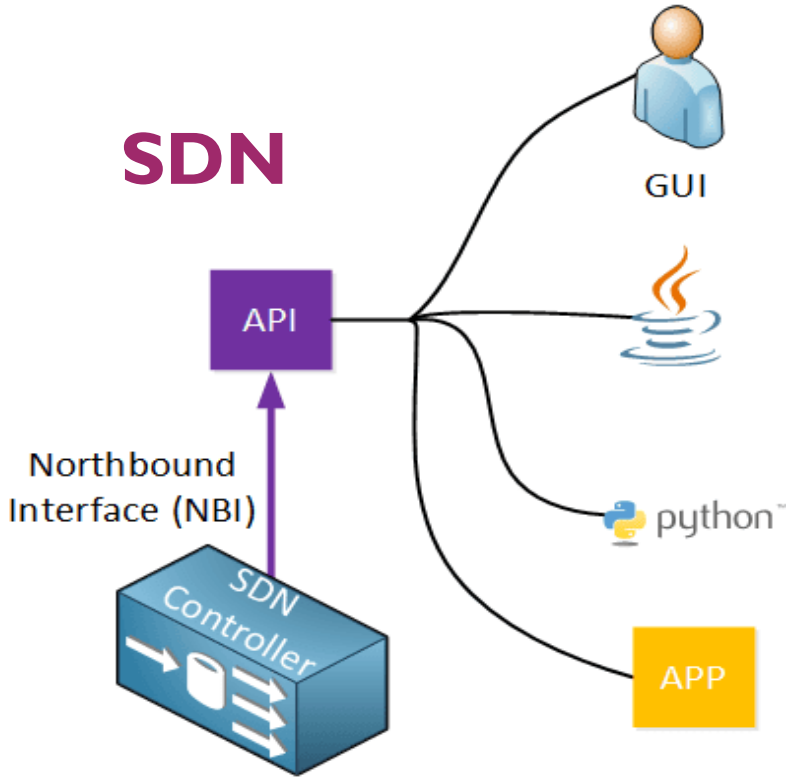
L'ARCHITECTURE SDN EST BASÉE SUR L'ÉVOLUTION DE L'ARCHITECTURE TRADITIONNELLE

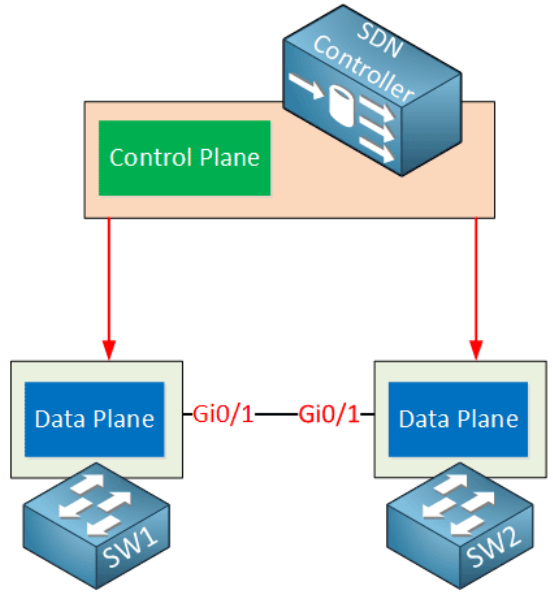


Traditional Networking

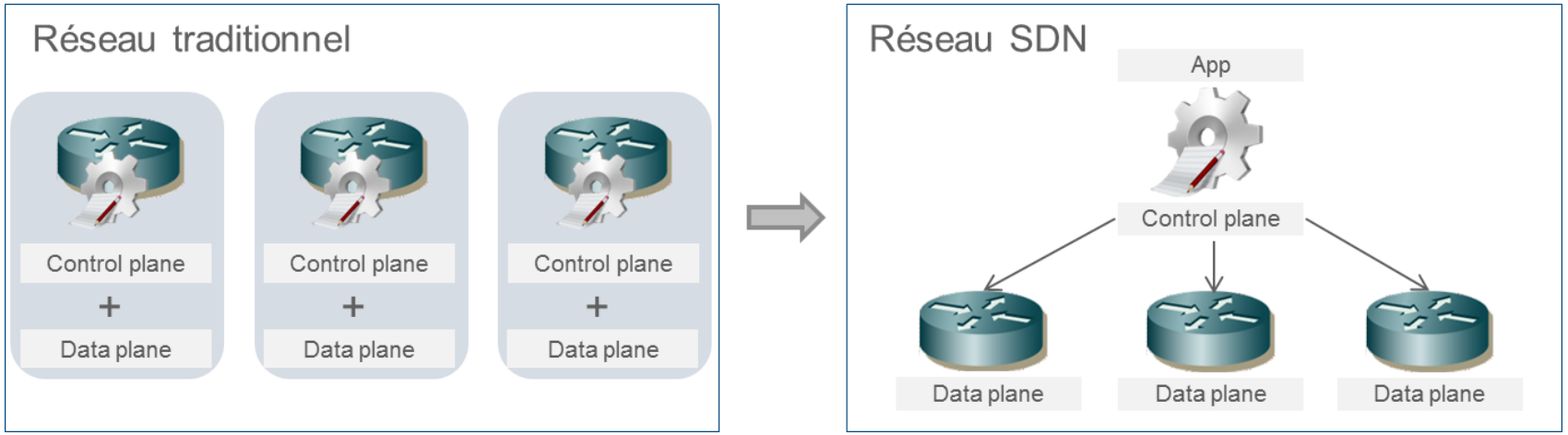


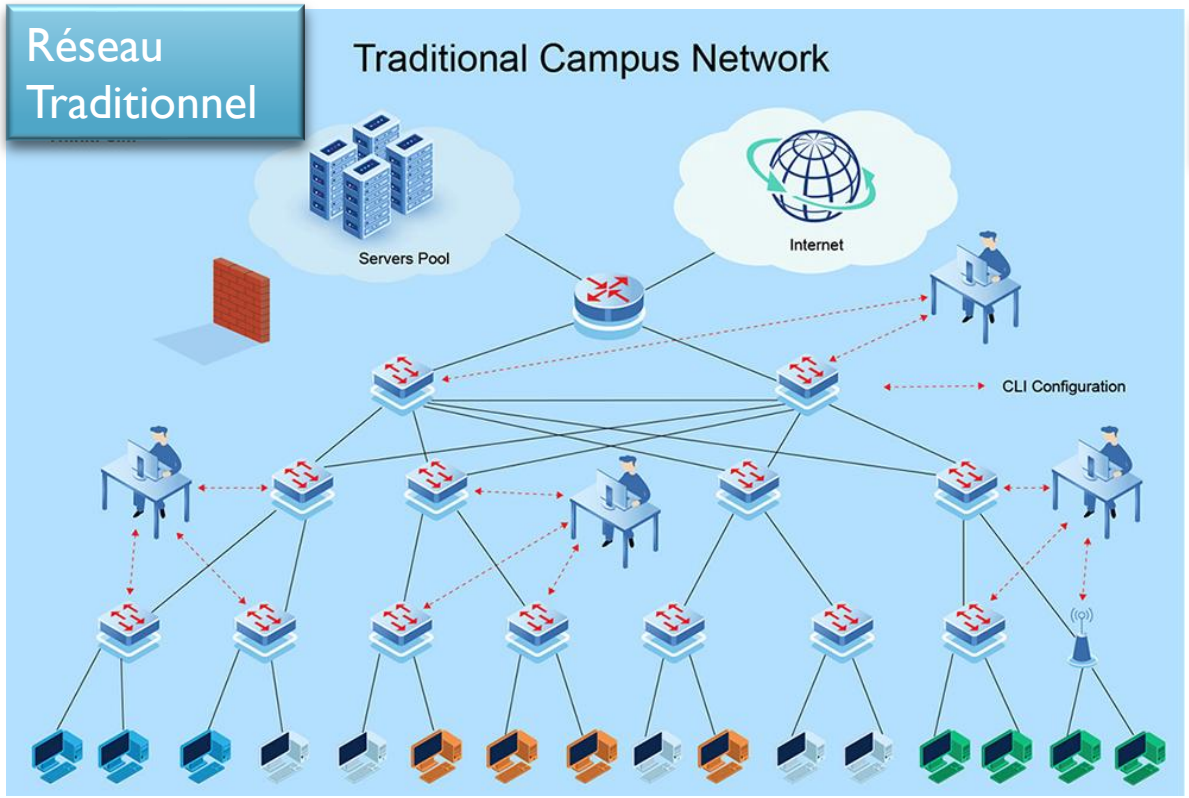
SDN

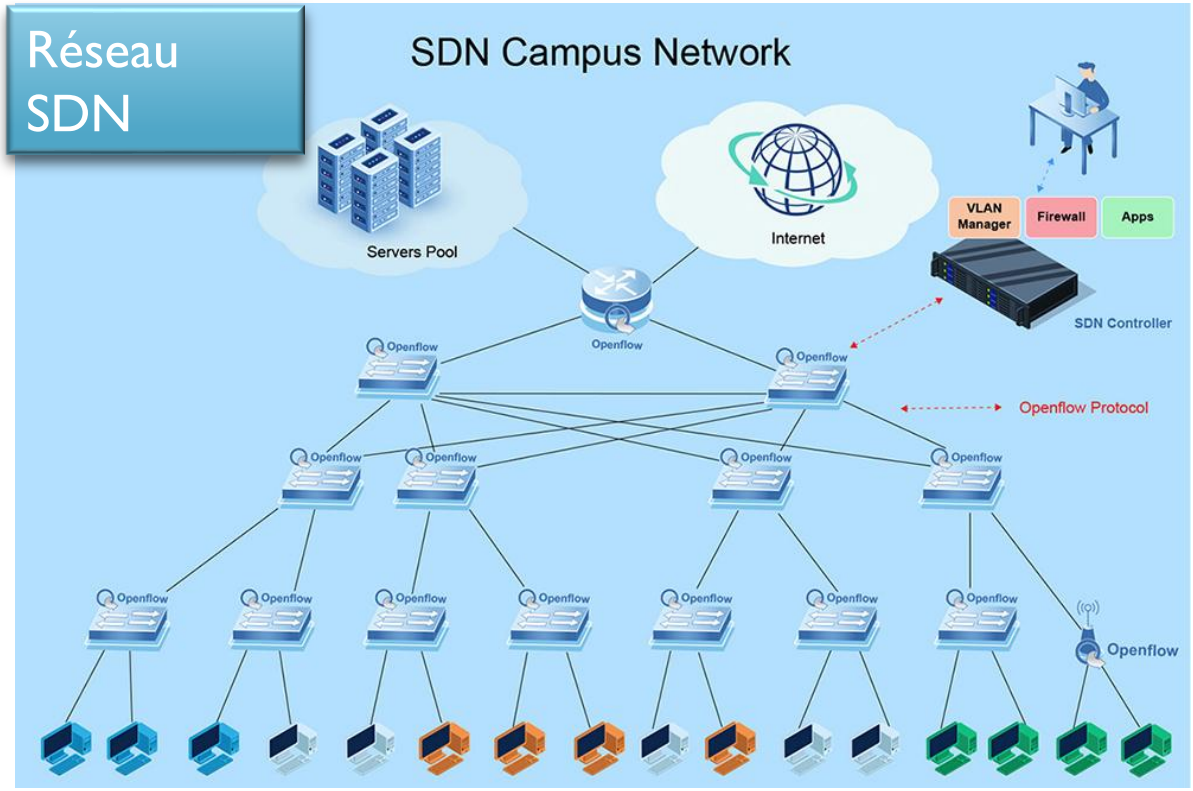




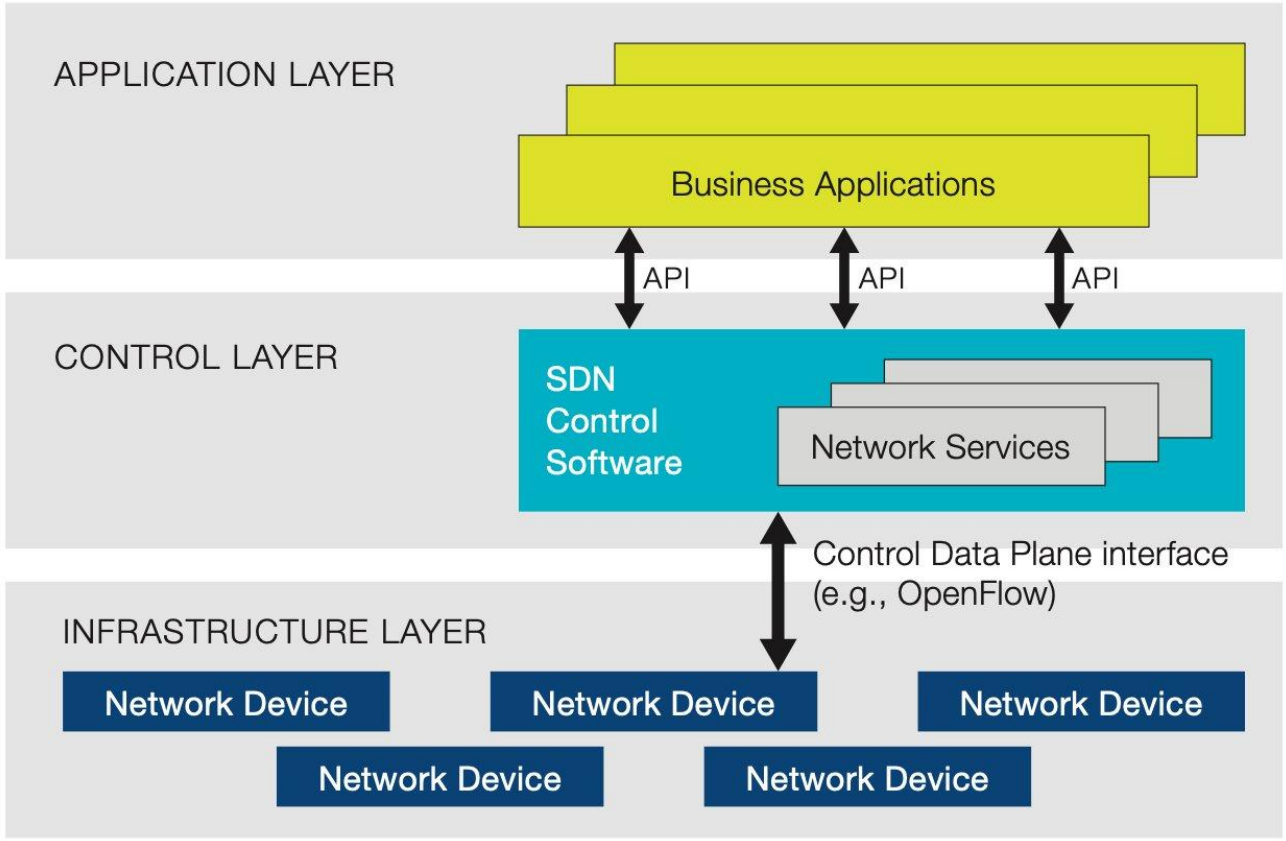
L’architecture traditionnelle et SDN

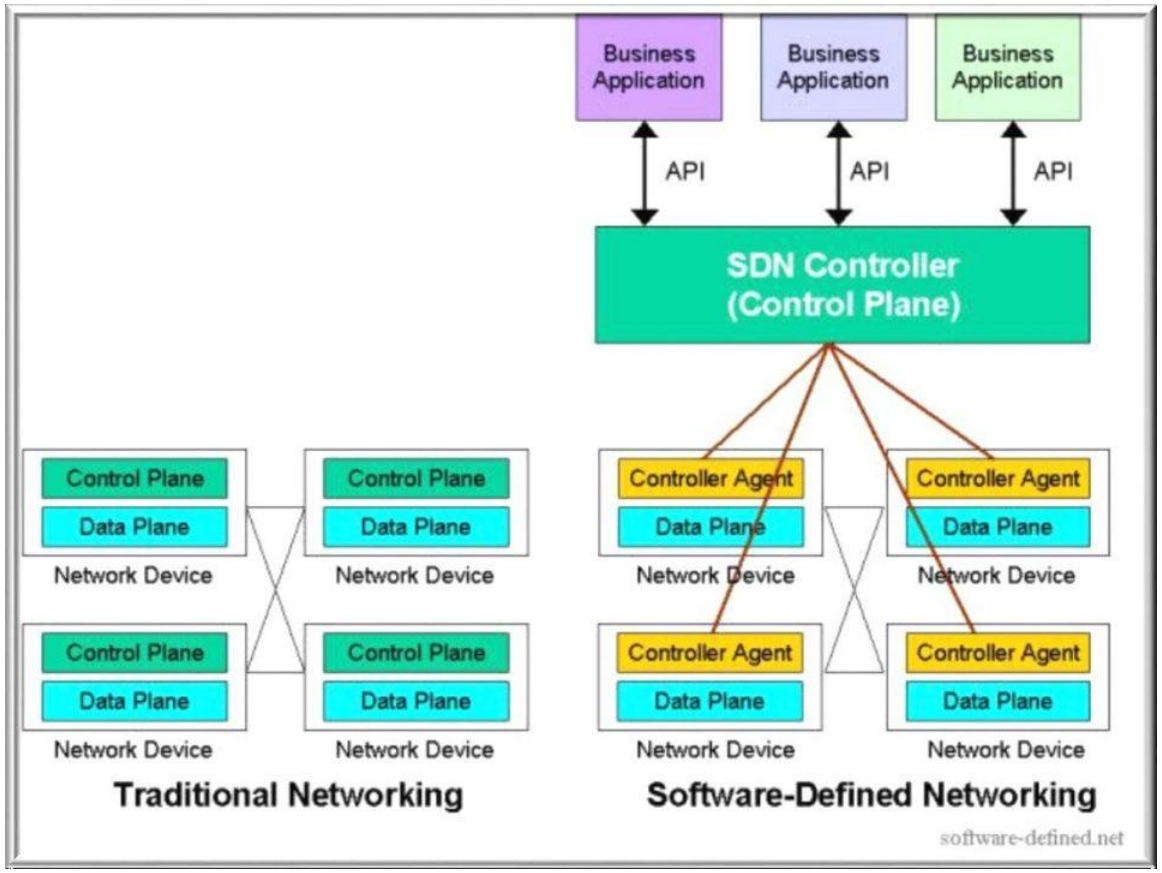




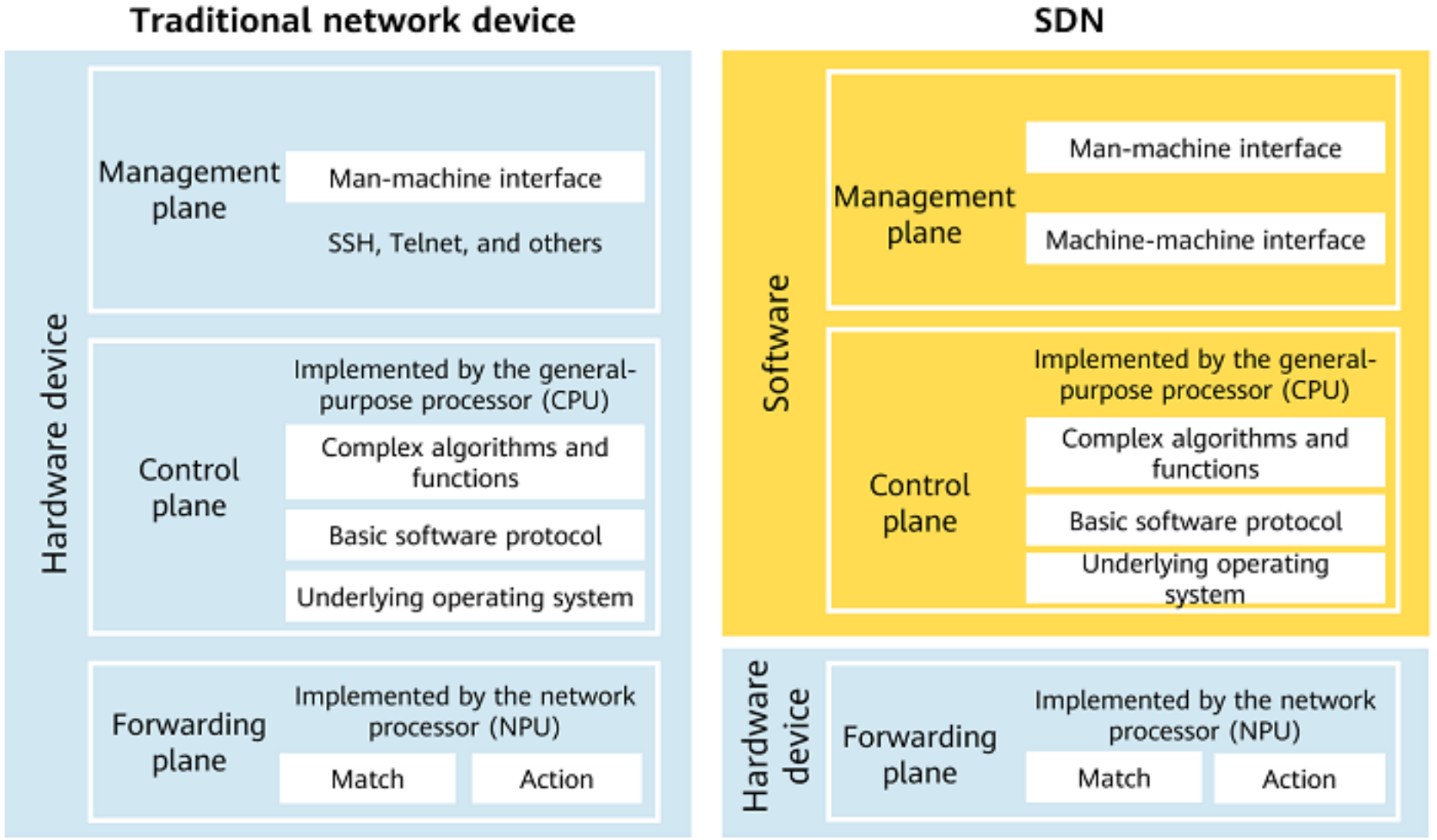


L’architecture traditionnelle: Problématique et solution



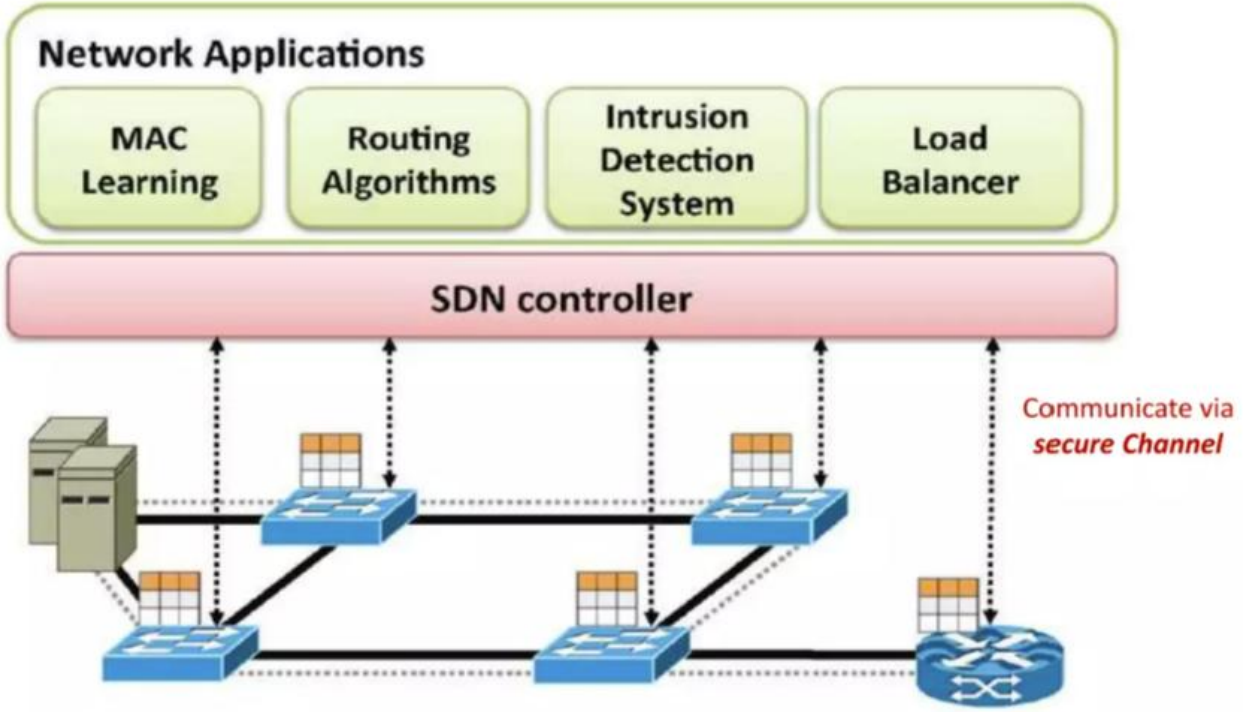


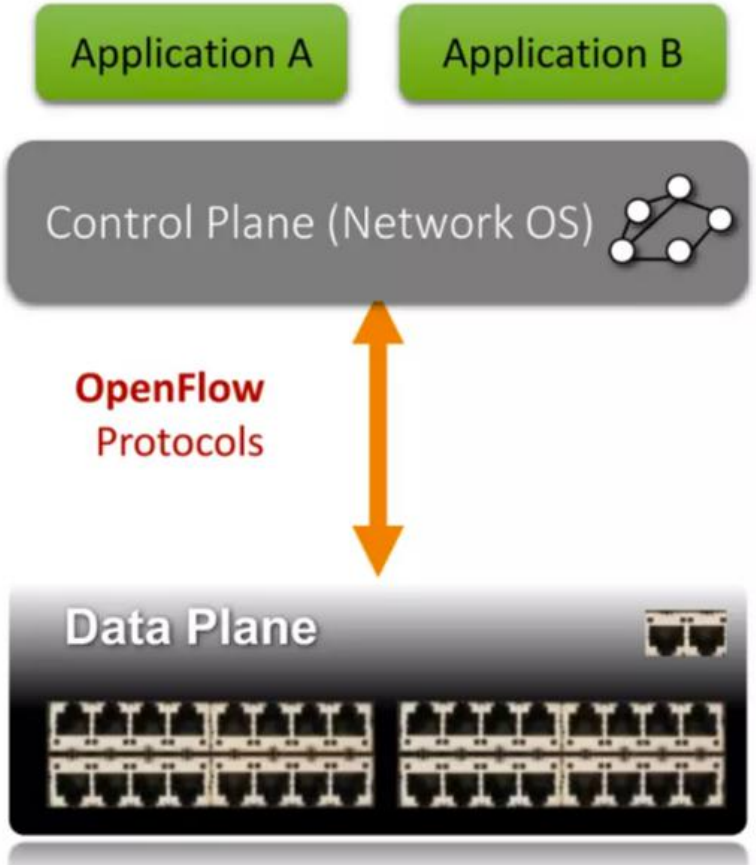
L’architecture traditionnelle et SDN



OpenFlow Protocol

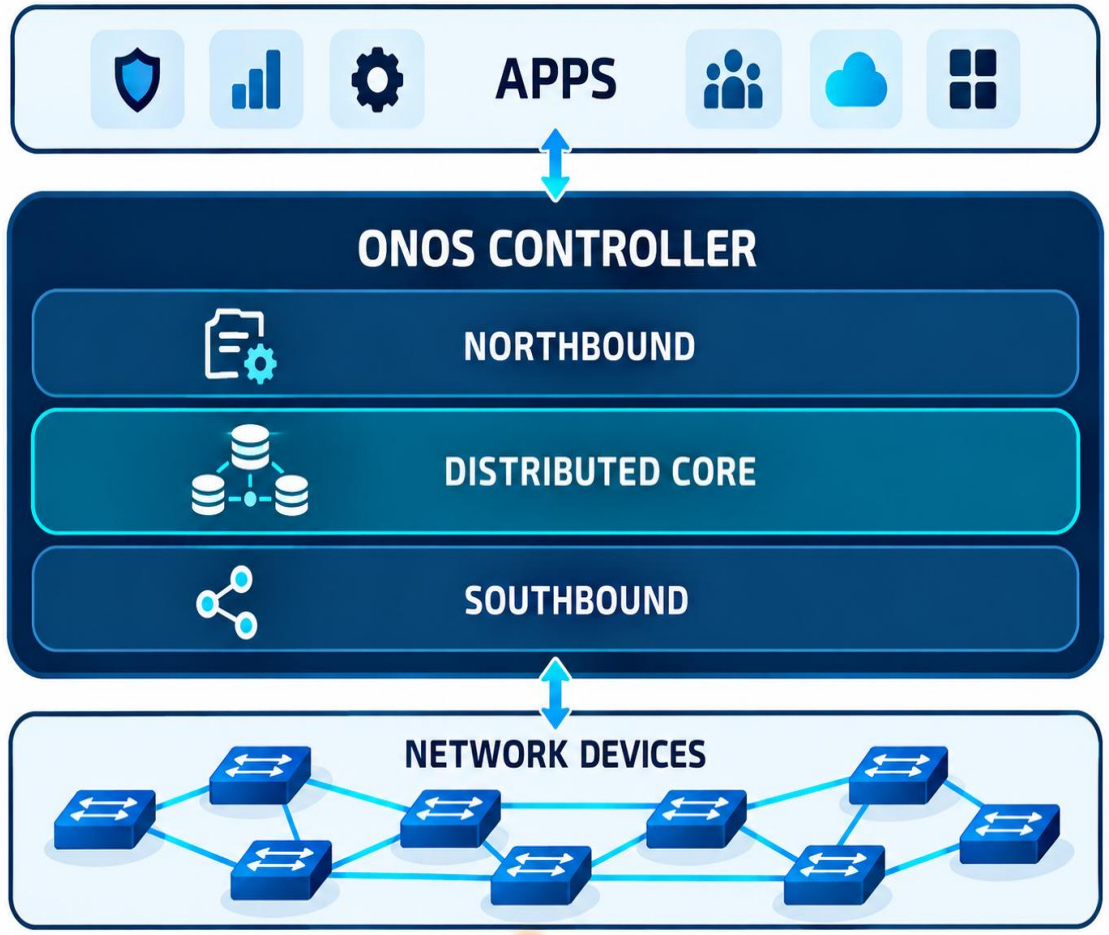
- Is communication interface between the control and data plane of an *SDN architecture*.





Du réseau traditionnel au SDN : où se place ONOS ?

Le contrôleur devient le point de décision entre les applications et les équipements réseau



1. Applications

Expriment un besoin réseau : connectivité, sécurité, mobilité, routage, automatisation.

2. Contrôleur ONOS

Maintient une vue globale du réseau, calcule les décisions et coordonne leur application.

3. Équipements

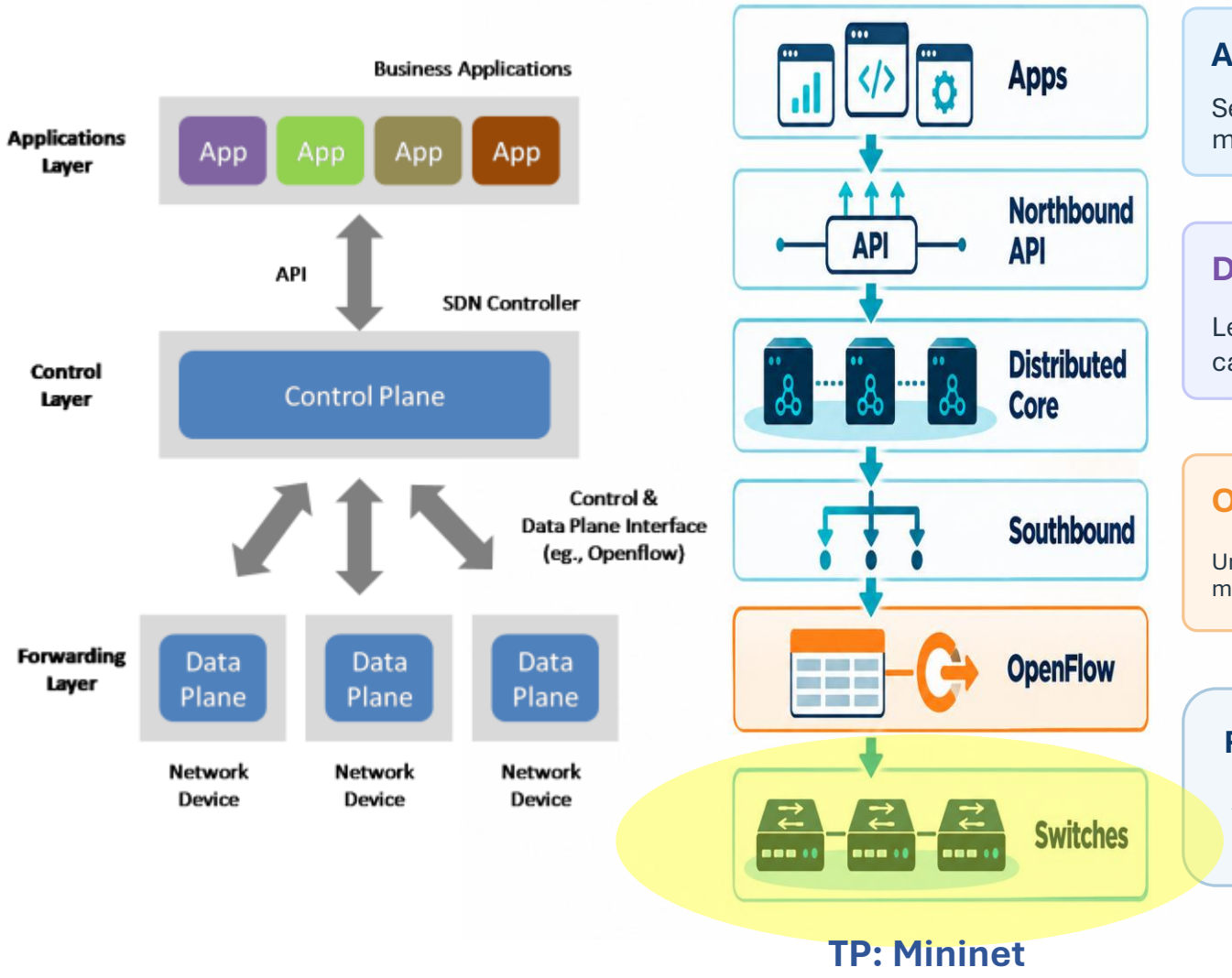
Les switches exécutent les règles reçues et transportent effectivement les paquets.

Idée clé : séparation du plan de contrôle et du plan de données

TP: Mininet

Architecture ONOS : rôle de chaque couche

Lecture du schéma de haut en bas



TP: Mininet

APPS

Services réseau : forwarding, mobilité, sécurité, intents...

Northbound

Interface avec les applications : intentions, politiques et APIs.

Distributed Core

Le « cerveau » : topologie, état, calculs, haute disponibilité.

Southbound

Découvrir, observer, programmer et configurer les équipements.

OpenFlow

Un protocole southbound pour installer / modifier / supprimer des règles.

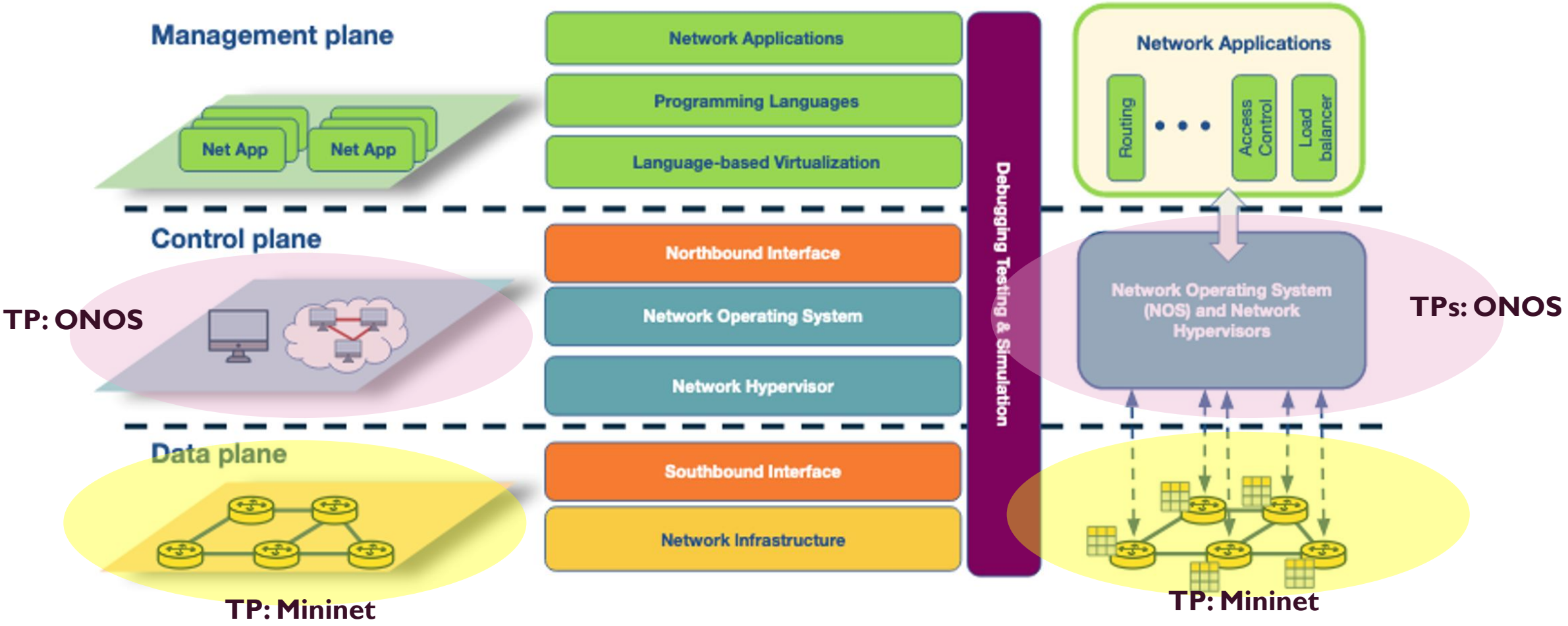
Switches

Plan de données : application des règles et transfert réel des paquets.

Résumé à faire retenir

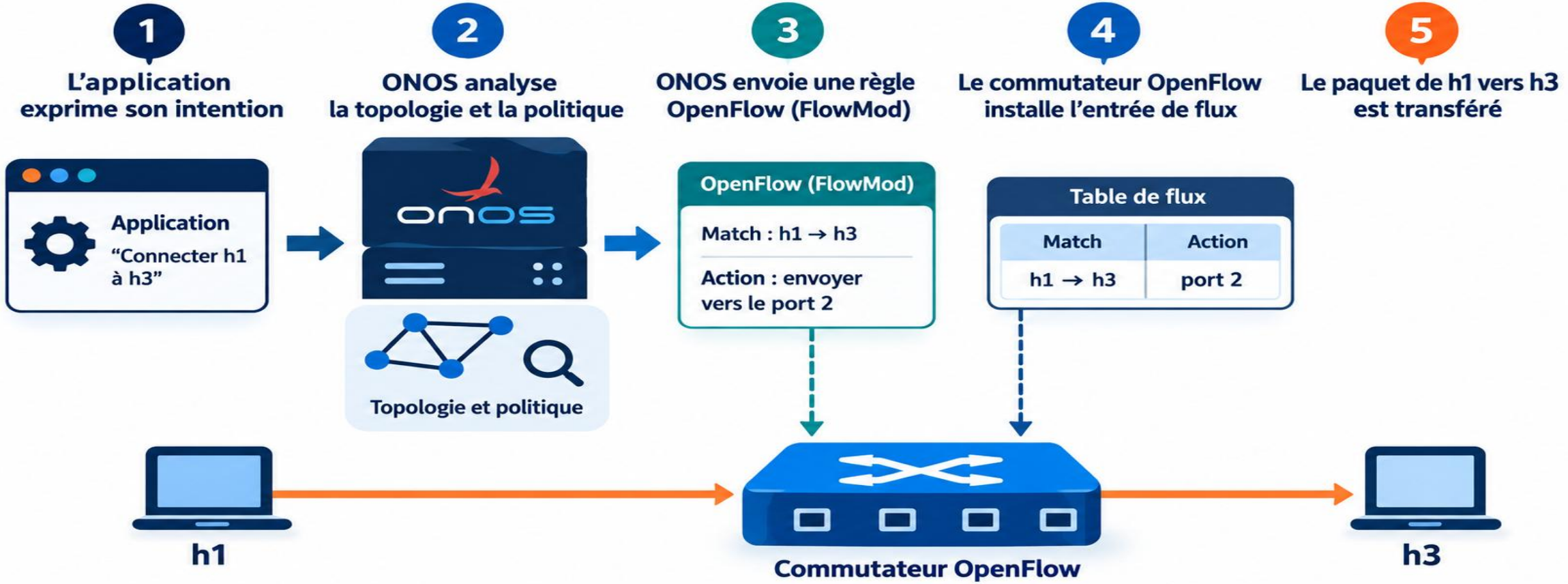
Applications demandent → ONOS décide → OpenFlow programme → Switches exécutent

L’architecture traditionnelle et SDN



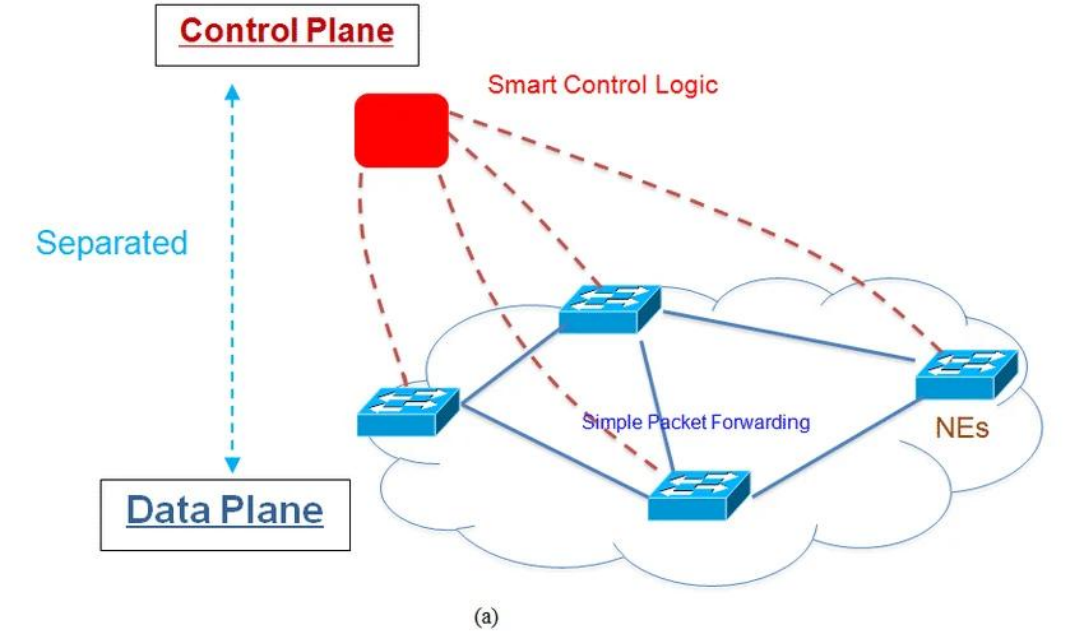
Exemple concret : de l'intention à la règle OpenFlow

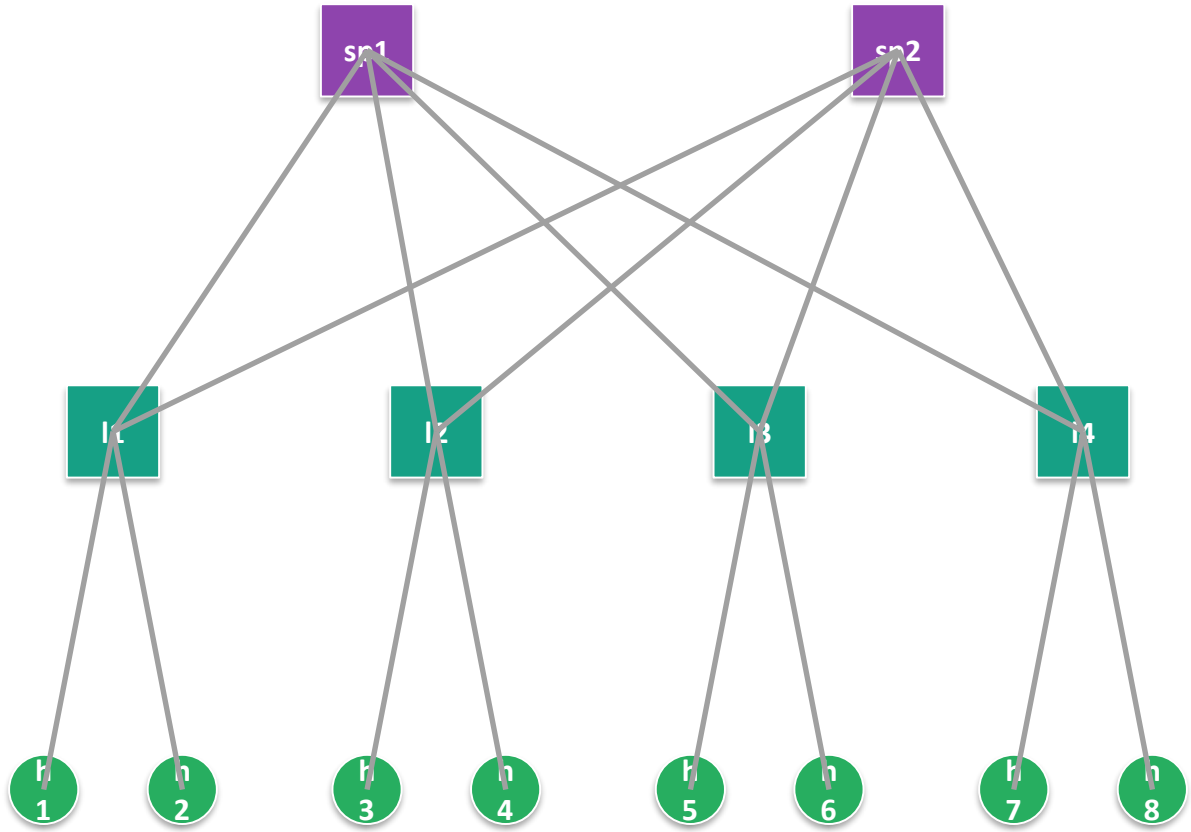
Même logique que dans le TP manuel — sauf qu'ici le contrôleur automatise la programmation des flux



Application demande → ONOS analyse → FlowMod OpenFlow → règle installée → trafic transféré

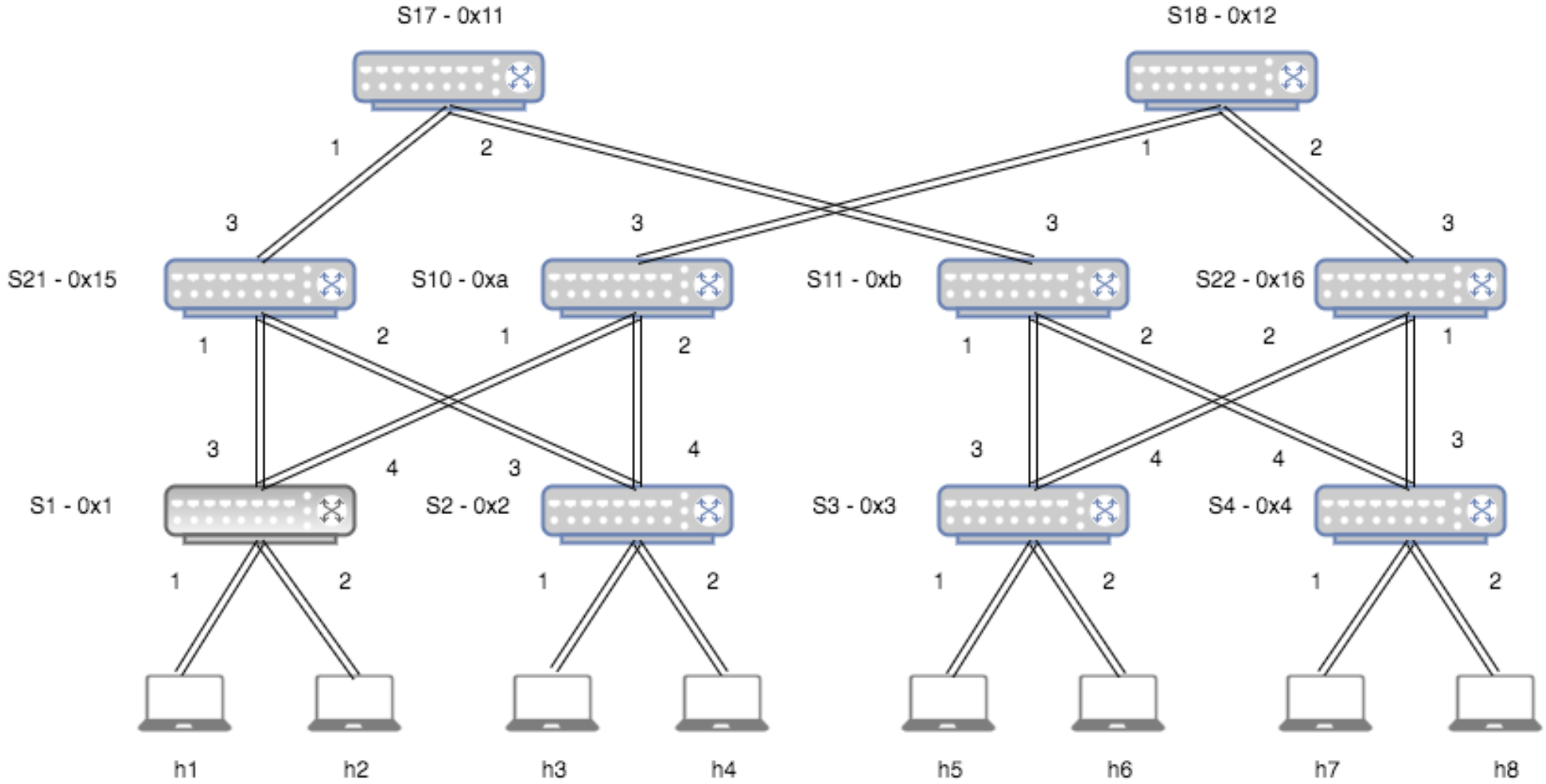
Control & Data planes separation and SDN architecture





SDN Load Balancing

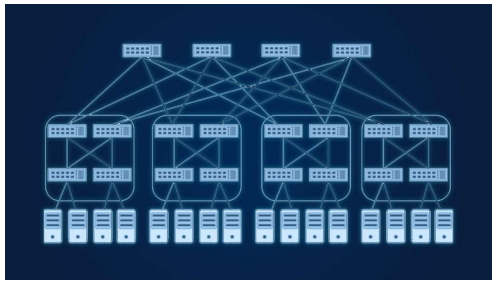
sudo mn --custom topology.py --topo mytopo --controller=remote,ip=127.0.0.1,port=6653



Exemple: Qu'est-ce qu'une topologie Fat-Tree ?

La **Fat-Tree** est une **topologie hiérarchique** inspirée des **arbres binaires inversés**, mais "grossie" en haut (d'où le nom *fat*).
Elle est conçue pour :

- offrir **plusieurs chemins redondants** entre serveurs ;
- **équilibrer la charge** du trafic ;
- et garantir une **grande bande passante agrégée** dans les datacenters.

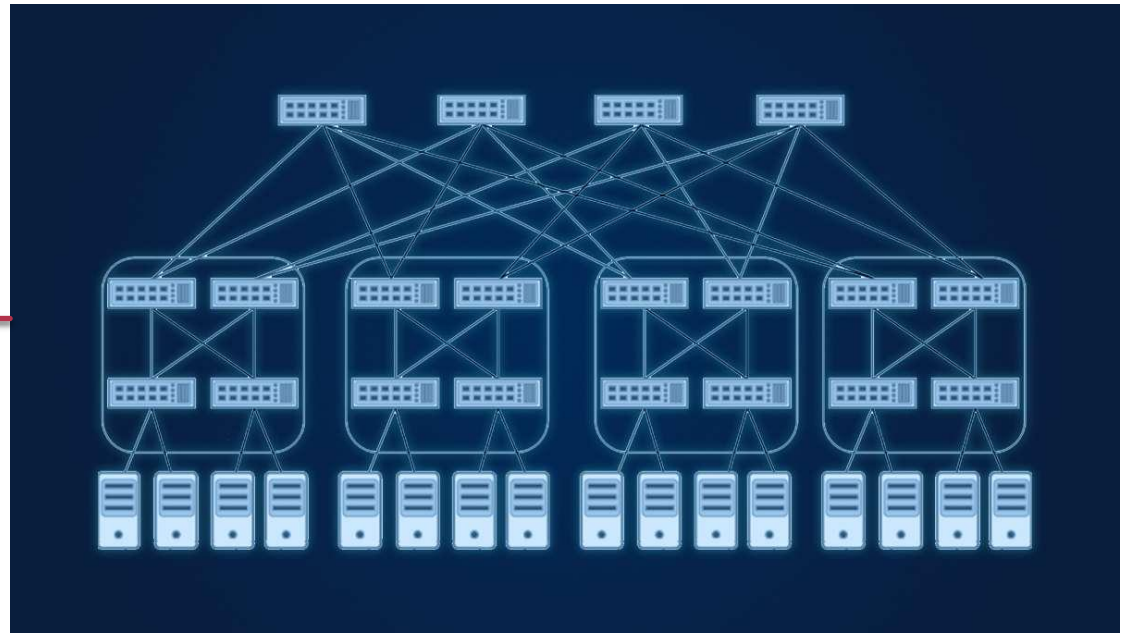


Structure typique :

- **Couche basse (Edge / ToR)** → connecte les hôtes (VMs, serveurs).
- **Couche intermédiaire (Aggregation)** → relie plusieurs switches Edge.
- **Couche haute (Core)** → relie toutes les agrégations entre elles.

Le résultat est **hautement maillé** :
chaque serveur a **plusieurs chemins possibles** vers les autres serveurs — ce qui maximise la **tolérance aux pannes et la bande passante**.

Exemple:Topologie Fat-Tree



Dans un réseau classique **(non SDN)** :

- Chaque **switch (s1–s12)** fonctionne **de manière autonome**.
- Il possède sa **propre table de commutation / routage**.
- Le **contrôle du trafic (plan de contrôle)** et le **transfert des paquets (plan de données)** sont **intégrés dans le même équipement**.
- Les décisions de routage sont faites localement avec des protocoles comme **STP, OSPF, RIP, BGP, VLANs**,

Conséquence concrète sur cette topologie :

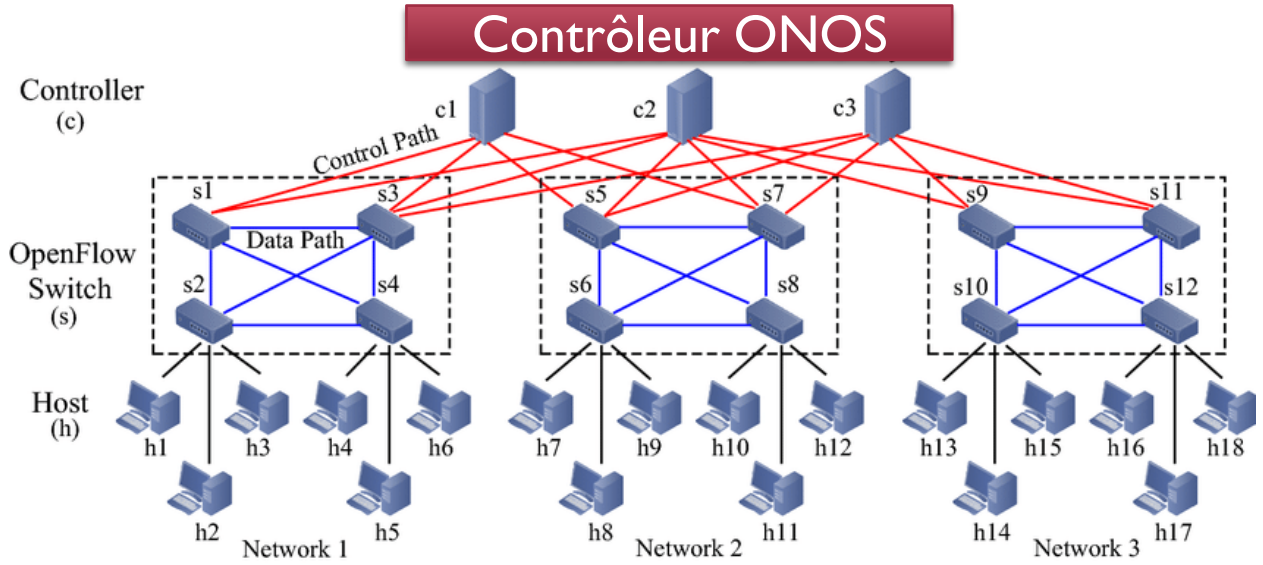
Si tu veux modifier la politique réseau (ex. bloquer un flux ou prioriser YouTube) → il faut **configurer manuellement** chaque switch.

Chaque domaine (Network1, Network2, Network3) est **indépendant**.

Il n'existe **aucune coordination globale** entre eux.

Impossible d'avoir une **vision d'ensemble** du trafic.

Avec un réseau SDN



Ici, on sépare **le plan de contrôle** et **le plan de données** :

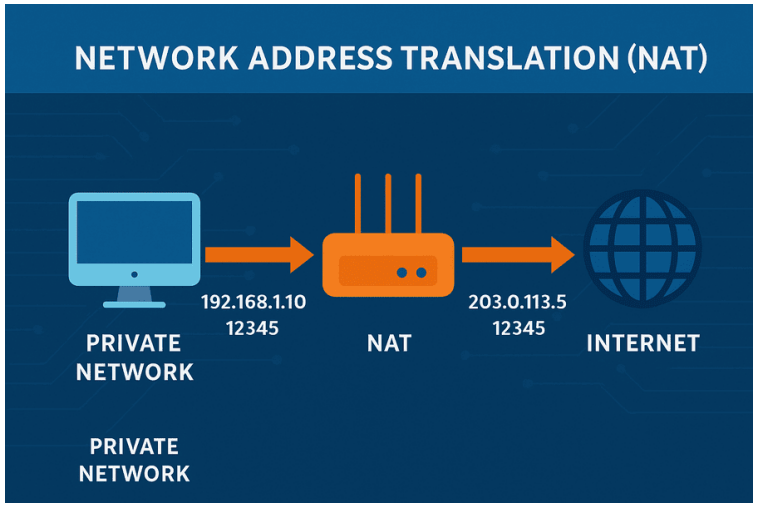
Élément	Rôle
Contrôleur SDN (c1–c3)	Cerveau central du réseau — prend les décisions de routage et applique les politiques.
Switch OpenFlow (s1–s12)	Exécute uniquement les ordres du contrôleur (plan de données).
Hosts (h1–h18)	Génèrent le trafic utilisateur.

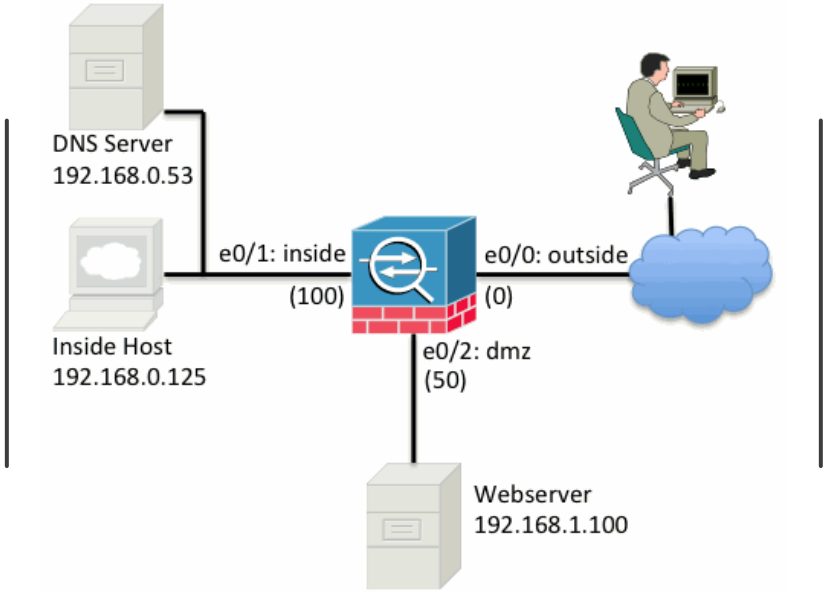
Aspect	Réseau traditionnel	Réseau SDN avec ONOS
Configuration	Chaque switch configuré manuellement (CLI, VLANs, OSPF…).	**ONOS** configure automatiquement tous les switches via **OpenFlow** et ses applications intégrées (fwd, intents, virtual network…).
Plan de contrôle	Distribué dans chaque switch → non coordonné.	**Centralisé, cohérent et programmable** : ONOS gère le plan de contrôle global et distribue les décisions aux switches.
Réaction aux pannes	Lente (reconvergence OSPF/STP, redémarrage manuel).	**Instantanée** : ONOS détecte la panne et **reprogramme dynamiquement les flux** pour rétablir le trafic.
Sécurité	ACL locales sur chaque switch, difficile à maintenir.	**Politiques globales et dynamiques** : ONOS applique des règles réseau via son **API REST** ou ses **intents** (par exemple, blocage d'un flux malveillant sur tout le réseau).
Supervision	Vision locale (chaque équipement isolé).	**Vision unifiée** du réseau : ONOS offre une **carte topologique globale**, des statistiques de flux et d'utilisation en temps réel.
Scalabilité	Difficile à étendre (configurations multiples, limites VLAN).	**Haute scalabilité** : ONOS est **distribué** (cluster de contrôleurs) et peut gérer **des milliers de switches et hôtes**.
Automatisation	Scripts CLI ou SNMP limités et non dynamiques.	**Automatisation complète** via **Python ou API REST d'ONOS** (provisionnement de flux, QoS, intents, isolation de réseaux virtuels).

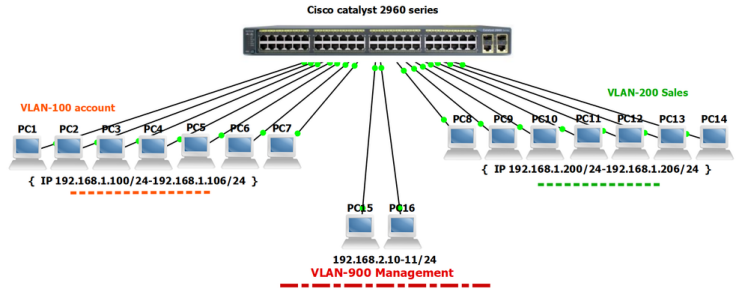
Pourquoi le SDN est crucial dans une Fat-Tree

Une topologie Fat-Tree est **puissante**, mais **très complexe à gérer** avec un réseau traditionnel.
Le SDN résout cette complexité grâce à la **programmabilité centralisée**.

Défi (réseau classique)	Solution (avec SDN)
Routage entre centaines de switches	Le **contrôleur SDN calcule dynamiquement** les meilleurs chemins (ECMP).
Équilibrage de charge (load balancing)	SDN applique des politiques d'**ECMP dynamique** ou de **flow hashing**.
Gestion des liens multiples	OpenFlow permet de **choisir la route par flux** (Flow entries).
Maintenance / panne	Le contrôleur **détecte la panne** et **reprogramme les flux** instantanément.
Sécurité inter-pod	Politiques globales appliquées depuis le contrôleur (micro-segmentation).







RAPPELS DE COURS

- **NAT**
- **ACL**
- **VLAN**

NAT statique (Static NAT)

Association fixe et permanente entre une adresse IP privée et une adresse IP publique (one-to-one).

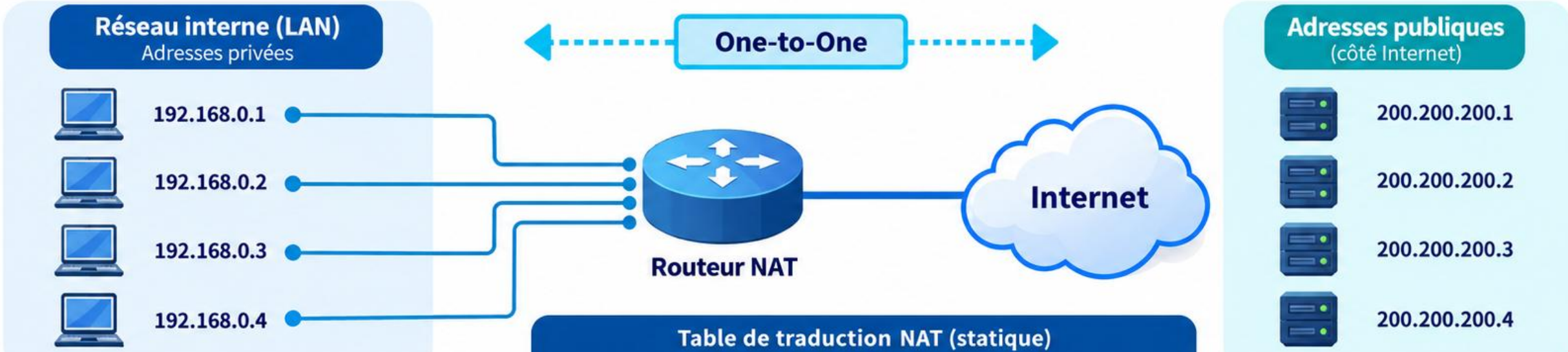


Table de traduction NAT (statique)

Inside Local IP (adresse privée)	Inside Global IP (adresse publique)
192.168.0.1	200.200.200.1
192.168.0.2	200.200.200.2
192.168.0.3	200.200.200.3
192.168.0.4	200.200.200.4

Correspondance fixe
Une adresse IP privée est toujours associée à la même adresse IP publique.

Utile pour publier un serveur
Permet de rendre un serveur interne (web, mail, etc.) accessible depuis Internet.

Consomme plus d'adresses publiques
Il faut une adresse IP publique par hôte interne.

Translation permanente
La correspondance est statique et reste active en permanence, indépendamment du trafic.

NAT dynamique (Dynamic NAT)

Association temporaire entre plusieurs adresses IP privées et un pool d'adresses IP publiques (**many-to-many with pool**).

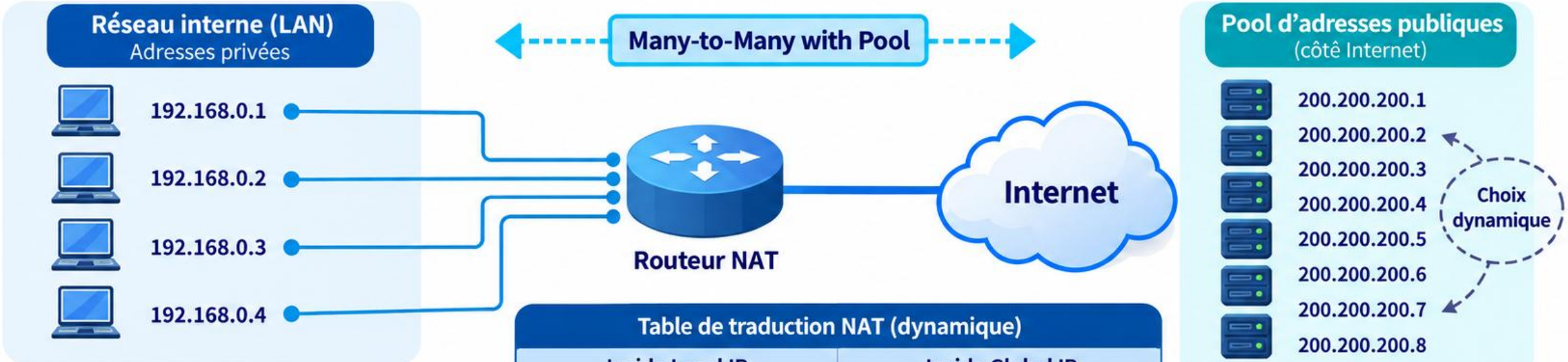


Table de traduction NAT (dynamique)

Inside Local IP (adresse privée)	Inside Global IP (adresse publique)
192.168.0.1	200.200.200.2
192.168.0.2	200.200.200.4
192.168.0.3	200.200.200.7
192.168.0.4	200.200.200.8

Pool public

Un ensemble d'adresses IP publiques est partagé entre plusieurs hôtes internes.

Attribution temporaire

Une adresse IP publique est utilisée seulement pendant la session (de manière dynamique).

Économie partielle

Nécessite moins d'adresses IP publiques que le NAT statique, mais plus que le PAT (NAT overload).

Dépend du pool

Si le pool d'adresses publiques est vide, les nouvelles traductions ne sont plus possibles.

PAT (NAT Overload)

Plusieurs adresses IP privées partagent une seule adressse IP publique grâce aux numéros de port **(many-to-one with ports).**

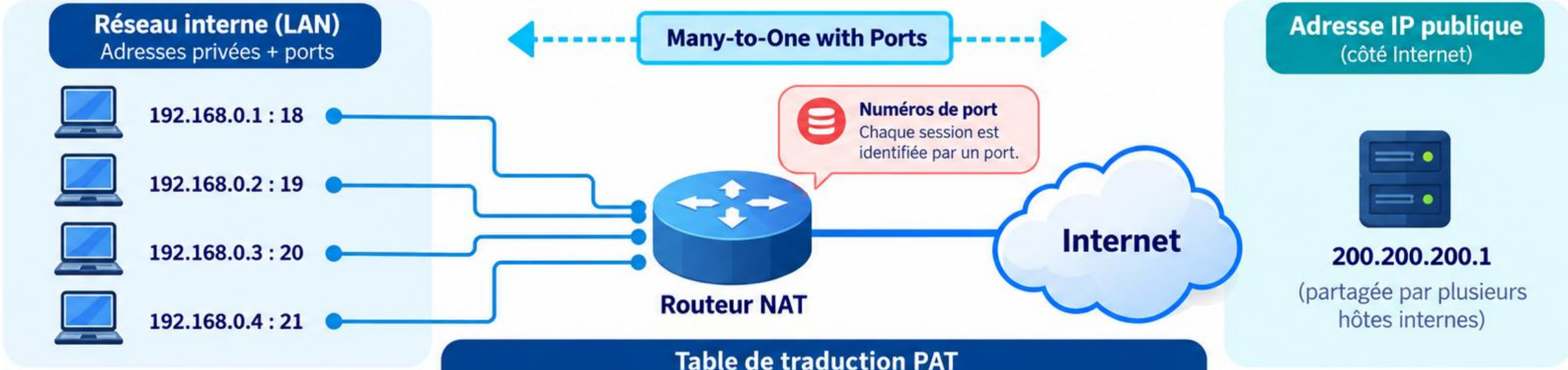


Table de traduction PAT

Inside Local IP:Port (adresse privée : port)	Inside Global IP:Port (adresse publique : port)
192.168.0.1 : 18	200.200.200.1 : 18
192.168.0.2 : 19	200.200.200.1 : 19
192.168.0.3 : 20	200.200.200.1 : 20
192.168.0.4 : 21	200.200.200.1 : 21

Une seule IP publique
Plusieurs hôtes internes partagent la même adresse IP publique.

Différenciation par ports
Chaque session est identifiée par un numéro de port unique.

Très économique
C'est la forme de NAT la plus courante pour l'accès Internet.

Limite
Le nombre de sessions dépend des ports disponibles.

NAT : Cisco classique vs pilotage SDN

Même objectif : traduire les adresses ; ce qui change, c'est l'endroit où l'on exprime la politique.

Réseau classique – Cisco IOS



192.168.1.0/24

PAT / NAT Overload sur Cisco

```
interface G0/0
 ip nat inside

interface G0/1
 ip nat outside

access-list 1 permit 192.168.1.0 0.0.0.255
ip nat inside source list 1 interface G0/1 overload
```

Politique configurée directement sur le routeur.

Réseau SDN – Contrôleur / API



Politique JSON (exemple conceptuel)

```
{
  "type": "pat",
  "inside_network": "192.168.1.0/24",
  "public_ip": "200.200.200.1",
  "action": "translate"
}
```

Le contrôleur reçoit l’intention puis programme les équipements conc

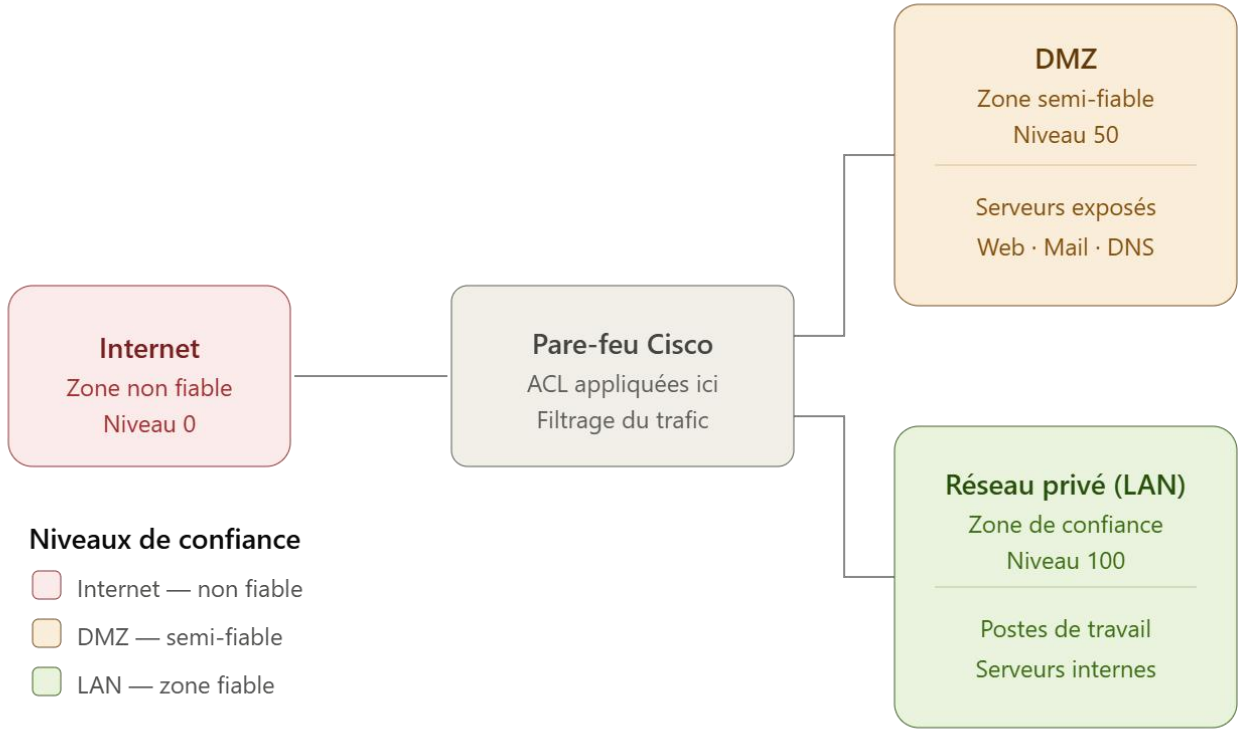
NB : il n’existe pas une API NAT universelle commune à tous les contrôleurs.

Cisco : configuration locale • SDN : politique centralisée puis déploiement automatisé

Définition & Objectifs des ACL

Access Control List — Contrôle du trafic réseau

Une ACL est un ensemble ordonné de règles (permit / deny) que le routeur Cisco applique aux paquets entrants ou sortants sur une interface. Elle filtre le trafic selon des critères : adresse IP source/destination, protocole, port TCP/UDP.



Filtrage du trafic

Autoriser ou bloquer des flux selon des critères précis (IP, port, protocole)

Sécurité périmétrique

Protéger les ressources internes contre les accès non autorisés depuis l'extérieur

Segmentation réseau

Contrôler les communications inter-VLAN, isoler les zones sensibles (DMZ, serveurs)

Conformité & audit

Appliquer les politiques de sécurité, réduire la surface d'attaque

Interprétation ligne par ligne :

- Ligne 1 : refuse tout trafic IP provenant de l'hôte unique `192.168.1.50` et destiné au réseau `192.168.2.0/24`. Le masque générique `0.0.0.255` couvre les 256 adresses du /24 (les bits à 1 sont ignorés).
- Ligne 2 : autorise tout le reste.

Cas 1 — Autoriser uniquement HTTP/HTTPS vers un serveur web

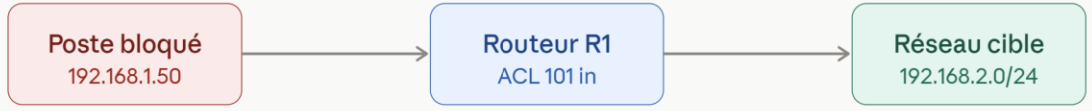



```
permit tcp any host 192.168.1.10 eq 80
permit tcp any host 192.168.1.10 eq 443
deny ip any host 192.168.1.10
permit ip any any
```

Interprétation ligne par ligne :

- Ligne 1 : autorise le trafic TCP de n'importe quelle source vers l'hôte `192.168.1.10` sur le port 80 (HTTP).
- Ligne 2 : autorise le TCP vers ce même hôte sur le port 443 (HTTPS).
- Ligne 3 : refuse **tout** autre trafic IP destiné à ce serveur (SSH, ICMP, FTP, etc.).
- Ligne 4 : autorise le reste du trafic vers toutes les autres destinations.

Cas 2 — Isoler un poste précis d'un sous-réseau




```
deny ip host 192.168.1.50 192.168.2.0 0.0.0.255
permit ip any any
```

ACL : configuration Cisco vs politique SDN

Même objectif : filtrer le trafic. Ce qui change, c’est la manière d’exprimer et de déployer la politique.

Réseau classique – Cisco IOS



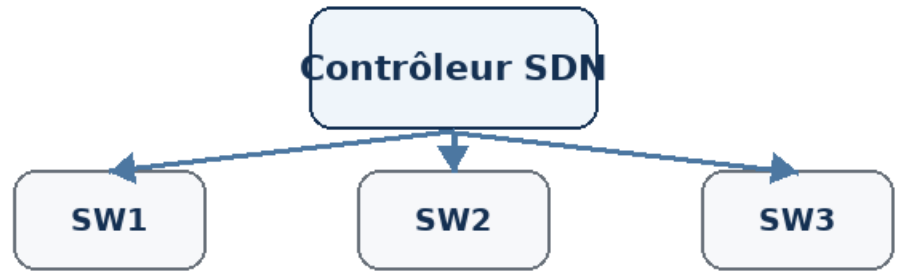
Exemple Cisco – ACL étendue

```
ip access-list extended WEB-HTTPS
 permit tcp 192.168.10.0 0.0.0.255 host 10.0.0.10 eq 443
 deny ip any any

interface GigabitEthernet0/1
 ip access-group WEB-HTTPS in
```

- Règle configurée localement sur l’équipement
- Très lisible pour un petit réseau
- Répétition nécessaire si la politique concerne plusieurs équipements

Réseau SDN – Contrôleur / API



Exemple JSON – intention de sécurité (conceptuelle)

```
{
  "src": "192.168.10.0/24",
  "dst": "10.0.0.10",
  "protocol": "tcp",
  "dst_port": 443,
  "action": "allow"
}
```

- Politique définie une fois puis diffusée par le contrôleur
- Cohérence sur plusieurs équipements
- API / automatisation adaptées aux environnements à grande échelle

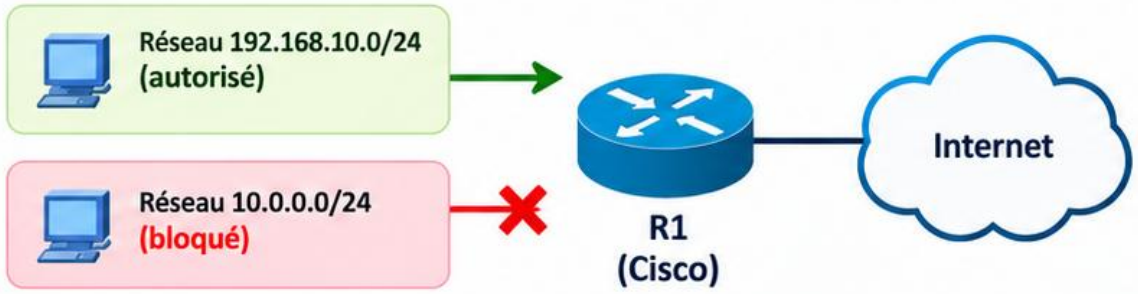
NB : le JSON ci-dessus est volontairement générique.

Cisco : ACL locale sur l’équipement • SDN : politique centralisée puis déploiement automatisé

Ce que le SDN apporte vraiment

Contrôle d'accès (ACL) : Réseau classique (Cisco) vs SDN

Réseau classique (Cisco)



Configuration ACL sur un routeur Cisco

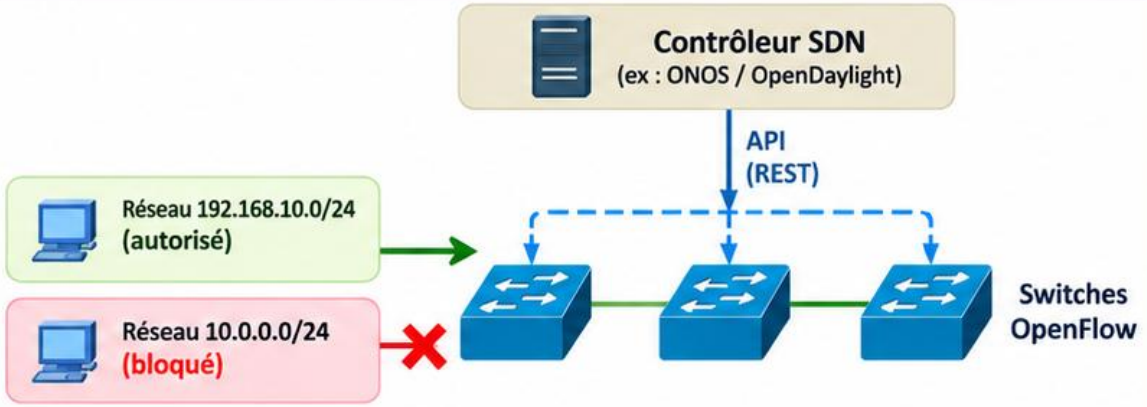
```
! Autoriser le trafic HTTP du réseau 192.168.10.00/24
! vers le réseau 10.0.0.0/24, bloquer le reste

ip access-list extended WEB
  permit tcp 192.168.10.0 0.0.0.255 10.0.0.0 0.0.0.255 eq 80
  deny  ip any any
!
interface GigabitEthernet0/1
 ip access-group WEB in
```

Principe

- Configuration sur chaque routeur/switch
- Règles définies localement (ACL)
- Gestion individuelle et manuelle
- Peu adapté aux grands réseaux (complexité)

SDN (OpenFlow / Contrôleur)



Configuration via API (exemple Python + REST)

```
import requests
controller = "http://10.0.0.1:8181"  # contrôleur SDN (ex : ONOS)
headers = {"Content-Type": "application/json"}

# Règle OpenFlow : autoriser HTTP du réseau 192.168.10.0/24
flow_allow = {
    "priority": 100,
    "match": {
        "in_port": 1,
        "eth_type": 0x0800,
        "ipv4_src": "192.168.10.0/24",
        "ipv4_dst": "10.0.0.0/24",
        "ip_proto": 6,
        "tcp_dst": 80
    },
    "actions": ["OUTPUT:2"]
}
requests.post(f"{controller}/onos/v1/flows", json=flow_allow, headers=headers)
```

Principe

- Règles définies une seule fois sur un contrôleur central
- Diffusion automatique vers tous les switches (OpenFlow)
- Gestion via une API (REST) avec Python, Java…
- Vision globale et cohérente des politiques
- Plus simple à administrer et à faire évoluer

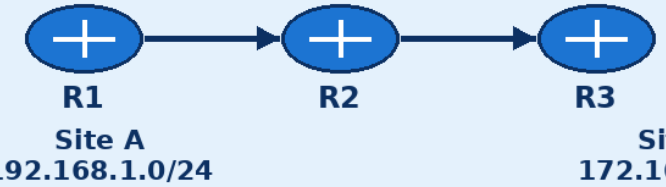
3) Même logique pour le routage et la segmentation

Le gain du SDN apparaît lorsque les changements deviennent fréquents, multi-sites ou multi-équipements.

2. Routage : Cisco classique vs SDN

Même objectif : acheminer les paquets, mais pas la même philosophie de contrôle

Réseau classique (Cisco IOS)

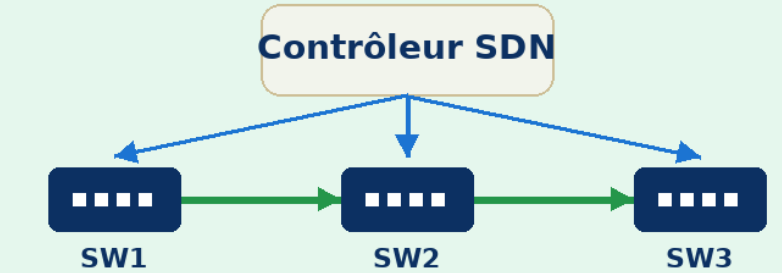


Configuration Cisco (exemple)

```
R1# ip route 172.16.2.0 255.255.255.0 10.0.0.2
R2# router ospf 1
    network 10.0.0.0 0.0.0.255 area 0
R3# ip route 192.168.1.0 255.255.255.0 10.0.0.2
```

- Configuration routeur par routeur
- Décisions distribuées / locales
- Maintenance plus lourde à grande échelle

SDN (Contrôleur + API / OpenFlow)



Politique via API (exemple Python)

```
policy = {
  "src": "192.168.1.0/24",
  "dst": "172.16.2.0/24",
  "preferred_path": ["SW1","SW2","SW3"]
}
requests.post(controller + "/flows", json=policy)
```

- Vision globale du réseau
- Déploiement automatique des règles
- Recalcul / bascule centralisés selon la politique

Routage : le contrôleur applique une politique de chemin, reroute en cas de panne, et garde une vue globale.

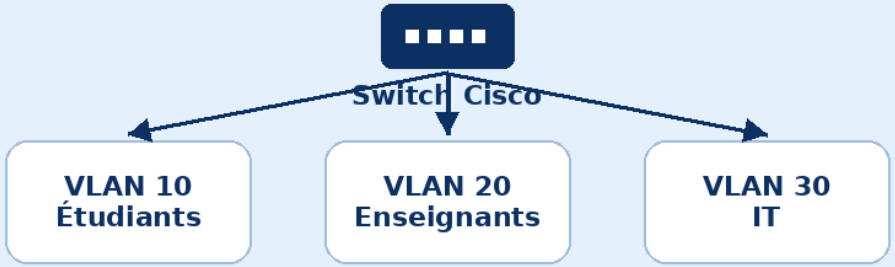
3) Même logique pour le routage et la segmentation

Le gain du SDN apparaît lorsque les changements deviennent fréquents, multi-sites ou multi-équipements.

3. VLAN : Cisco classique vs SDN

Même segmentation logique, mais configuration locale ou orchestration centralisée

Réseau classique (Cisco IOS)

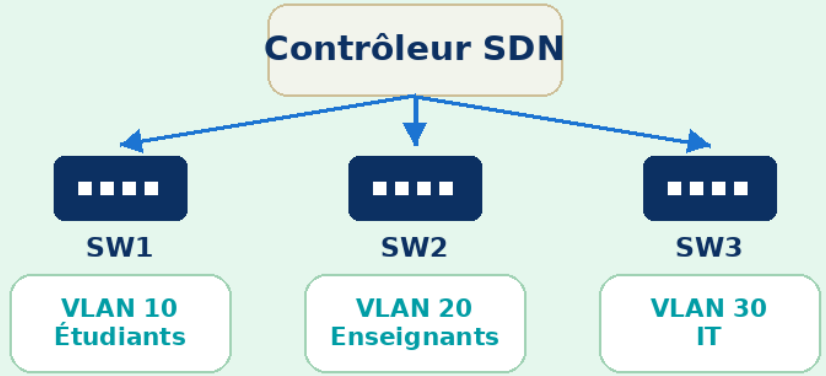


Configuration Cisco (CLI)

```
vlan 10
 name ETUDIANTS
interface GigabitEthernet0/1
 switchport mode access
 switchport access vlan 10
interface GigabitEthernet0/24
 switchport mode trunk
```

- Création VLAN switch par switch
- Affectation port par port
- Risque d'incohérence entre équipements

SDN (Contrôleur + API)



Création / affectation via API (exemple Python)

```
vlans = [
 {"id":10,"name":"ETUDIANTS"},
 {"id":20,"name":"ENSEIGNANTS"},
 {"id":30,"name":"IT"} ]
requests.post(controller + "/vlans", json=vlans)
requests.post(controller + "/ports/1", json={"vlan":10})
```

- Politique définie une fois
- Application cohérente sur tous les switches
- Automatisation et orchestration centralisées

VLAN / segmentation : une seule intention peut être déployée de façon cohérente sur l'ensemble du domaine.

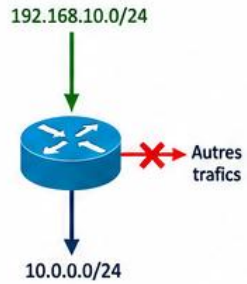
1 Contrôle d'accès (ACL) : Cisco vs SDN

Réseau classique (Cisco)

Configuration ACL sur un routeur/switch Cisco

```
! Autoriser le trafic HTTP du réseau 192.168.10.0/24
! vers le réseau 10.0.0.0/24

ip access-list extended WEB
  permit tcp 192.168.10.0 0.0.0.255 10.0.0.0 0.0.0.255 eq 80
  deny ip any any
!
interface GigabitEthernet0/1
  ip access-group WEB in
```

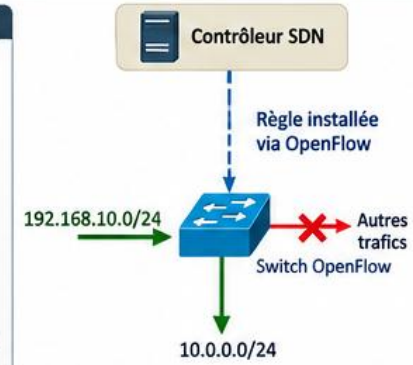


Même politique, mais pilotée centralement

SDN (OpenFlow / Contrôleur)

Configuration via API (exemple Python)

```
import requests
controller = "http://10.0.0.1:8080"
headers = {"Content-Type": "application/json"}
flow = {
    "priority": 100,
    "match": {
        "eth_type": 0x0800,
        "ip4proto": 6,
        "ipv4_src": "192.168.10.0/24",
        "ipv4_dst": "10.0.0.0/24",
        "tcp_dst": 80
    },
    "actions": ["OUTPUT:port2"]
}
requests.post(f"{controller}/stats/flowentry/add",
              json=flow, headers=headers)
```



2 Routage : Cisco vs SDN

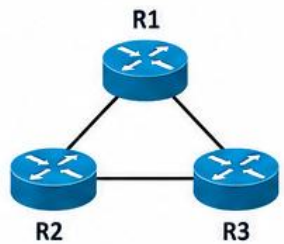
Réseau classique (Cisco)

Configuration de routage statique (Cisco)

```
! Route vers le réseau 10.0.1.0/24 via 192.168.1.2
ip route 10.0.1.0 255.255.255.0 192.168.1.2
```

Ou avec un protocole dynamique (ex : OSPF)

```
router ospf 1
  network 192.168.0.0 0.0.255.255 area 0
  network 10.0.0.0 0.0.255.255 area 0
```

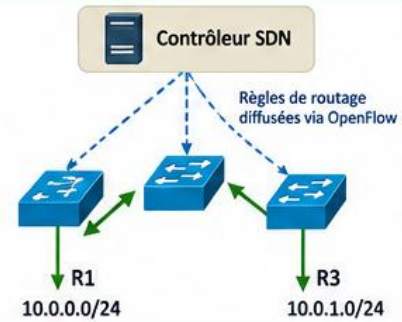


Même objectif : acheminer les paquets, mais les décisions sont centralisées

SDN (Contrôleur)

Installation de routes via API (exemple Python)

```
import requests
controller = "http://10.0.0.1:8080"
headers = {"Content-Type": "application/json"}
flow = {
    "priority": 100,
    "match": {
        "eth_type": 0x0800,
        "ipv4_dst": "10.0.1.0/24"
    },
    "actions": ["OUTPUT:port3"]  # port vers R3
}
requests.post(f"{controller}/stats/flowentry/add",
              json=flow, headers=headers)
```

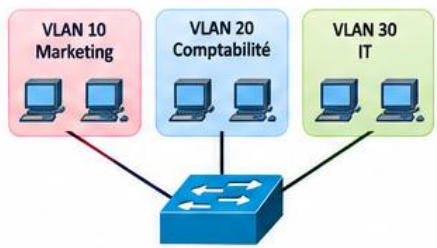


3 VLAN : Cisco vs SDN

Réseau classique (Cisco)

Configuration VLAN sur un switch Cisco

```
vlan 10
  name Marketing
vlan 20
  name Comptabilité
vlan 30
  name IT
!
interface GigabitEthernet0/1
  switchport mode access
  switchport access vlan 10
!
interface GigabitEthernet0/24
  switchport mode trunk
  switchport trunk allowed vlan 10,20,30
```



Même segmentation, mais gérée par une politique centralisée

SDN (Contrôleur)

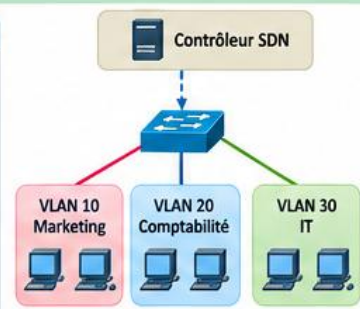
Création de VLANs via API (exemple Python)

```
import requests
controller = "http://10.0.0.1:8080"
headers = {"Content-Type": "application/json"}

# Créer les VLANs
vlan10 = {"vlan_id": 10, "name": "Marketing"}
vlan20 = {"vlan_id": 20, "name": "Comptabilite"}
vlan30 = {"vlan_id": 30, "name": "IT"}

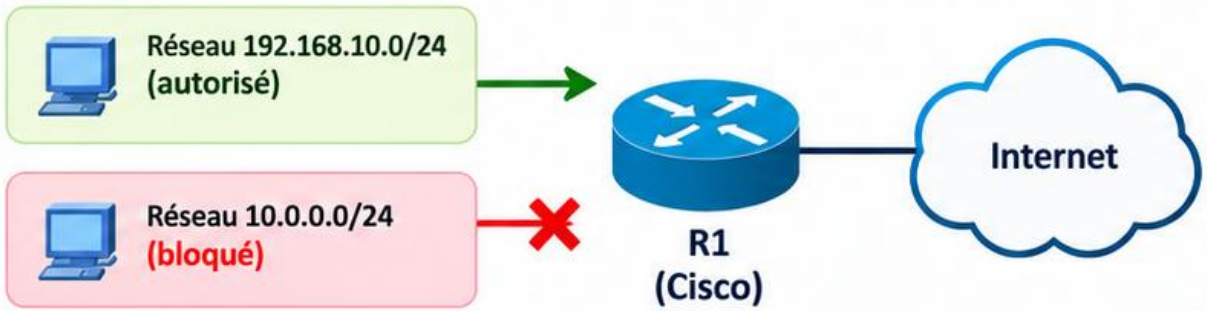
requests.post(f"{controller}/v1/vlans", json=vlan10, headers=headers)
requests.post(f"{controller}/v1/vlans", json=vlan20, headers=headers)
requests.post(f"{controller}/v1/vlans", json=vlan30, headers=headers)

# Associer un port à un VLAN
port_config = {"port": 1, "vlan_id": 10}
requests.post(f"{controller}/v1/ports", json=port_config, headers=headers)
```



Contrôle d'accès (ACL) : Réseau classique (Cisco) vs SDN

Réseau classique (Cisco)



Configuration ACL sur un routeur Cisco

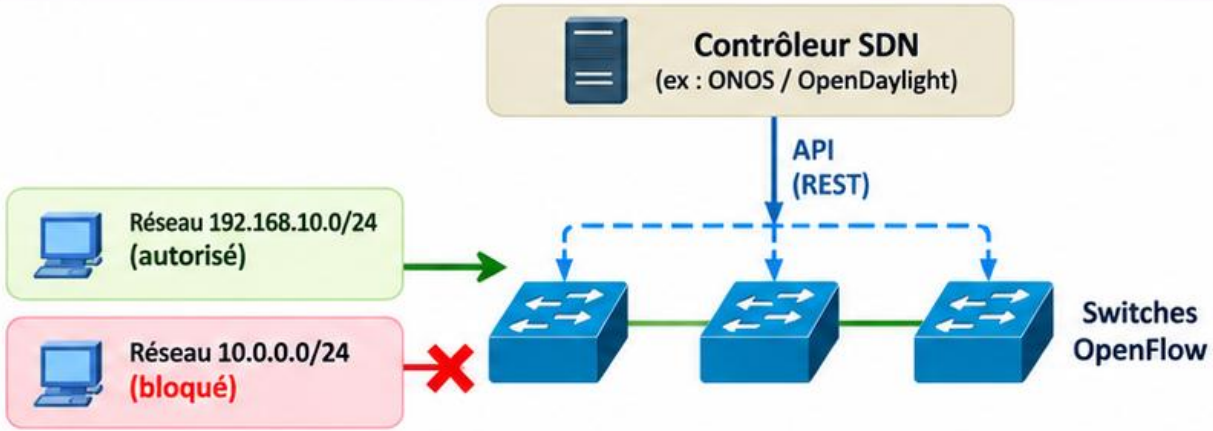
```
! Autoriser le trafic HTTP du réseau 192.168.10.00/24
! vers le réseau 10.0.0.0/24, bloquer le reste

ip access-list extended WEB
  permit tcp 192.168.10.0 0.0.0.255 10.0.0.0 0.0.0.255 eq 80
  deny  ip any any
!
interface GigabitEthernet0/1
 ip access-group WEB in
```

Principe

- Configuration sur chaque routeur/switch
- Règles définies localement (ACL)
- Gestion individuelle et manuelle
- Peu adapté aux grands réseaux (complexité)

SDN (OpenFlow / Contrôleur)



Configuration via API (exemple Python + REST)

```python
import requests
controller = "http://10.0.0.1:8181"  # contrôleur SDN (ex : ONOS)
headers = {"Content-Type": "application/json"}

# Règle OpenFlow : autoriser HTTP du réseau 192.168.10.0/24
flow_allow = {
    "priority": 100,
    "match": {
        "in_port": 1,
        "eth_type": 0x0800,
        "ipv4_src": "192.168.10.0/24",
        "ipv4_dst": "10.0.0.0/24",
        "ip_proto": 6,
        "tcp_dst": 80
    },
    "actions": ["OUTPUT:2"]
}
requests.post(f"{controller}/onos/v1/flows", json=flow_allow, headers=headers)
```

Principe

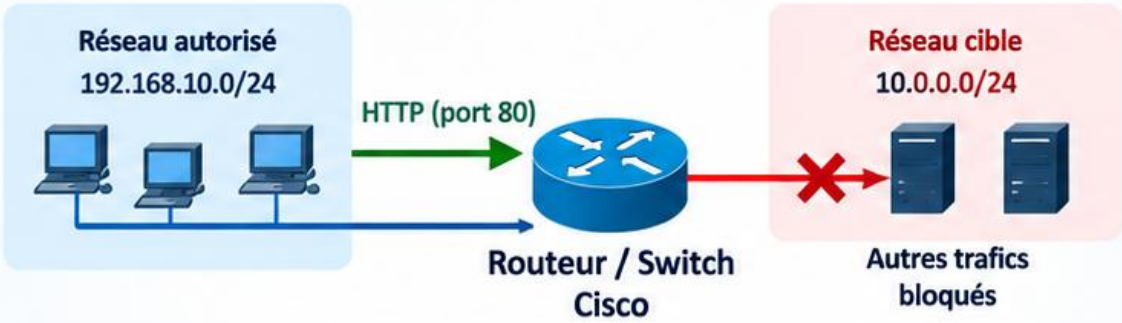
- Règles définies une seule fois sur un contrôleur central
- Diffusion automatique vers tous les switches (OpenFlow)
- Gestion via une API (REST) avec Python, Java…
- Vision globale et cohérente des politiques
- Plus simple à administrer et à faire évoluer

1. Contrôle d'accès (ACL) : Réseau classique (Cisco) vs SDN

Réseau classique (Cisco)

Configuration ACL sur un routeur/switch Cisco (CLI)

```
! ACL pour autoriser HTTP depuis 192.168.10.0/24
! et bloquer le reste vers 10.0.0.0/24
ip access-list extended WEB
  permit tcp 192.168.10.0 0.0.0.255 10.0.0.0 0.0.0.255 eq 80
  deny ip any any
!
interface GigabitEthernet0/1
  ip access-group WEB in
```



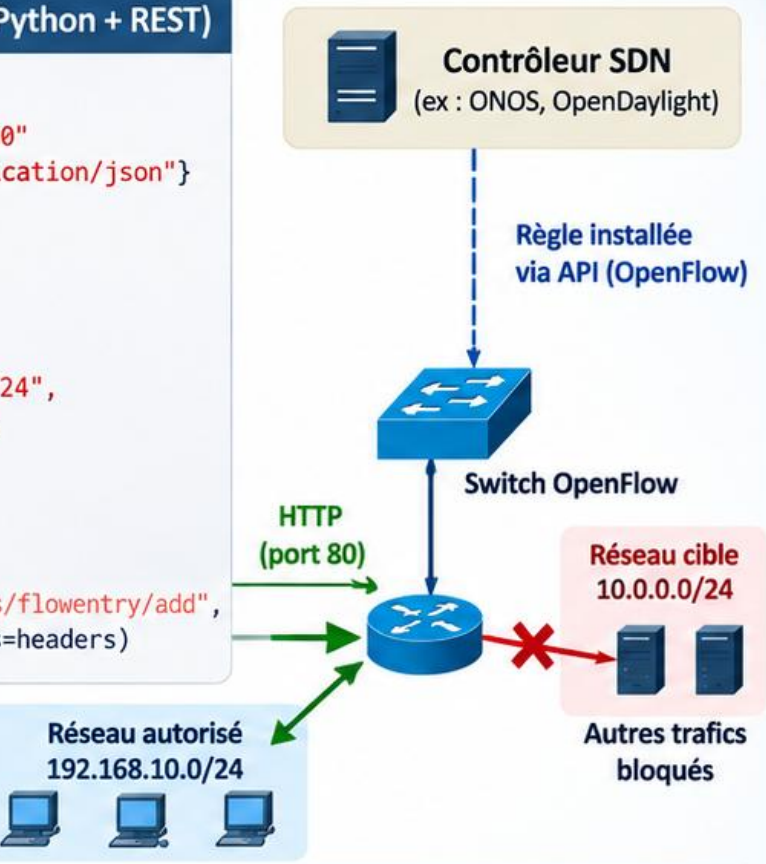
Caractéristiques

- Configuration équipement par équipement
- Règles locales sur chaque routeur/switch
- Gestion manuelle et individuelle
- Peu flexible à grande échelle

SDN (OpenFlow / Contrôleur)

Configuration via API (exemple Python + REST)

```
import requests
controller = "http://10.0.0.1:8080"
headers = {"Content-Type": "application/json"}
flow = {
   "priority": 100,
   "match": {
      "eth_type": 0x0800,
      "ip_proto": 6,
      "ipv4_src": "192.168.10.0/24",
      "ipv4_dst": "10.0.0.0/24",
      "tcp_dst": 80
   },
   "actions": ["OUTPUT:port2"]
}
requests.post(f"{controller}/stats/flowentry/add",
              json=flow, headers=headers)
```



Caractéristiques

- Politique définie de manière centralisée
- Règles installées dynamiquement via le contrôleur
- Programmation via API (Python, REST, OpenFlow)
- Application automatique sur plusieurs équipements
- Plus flexible et plus facile à gérer à grande échelle

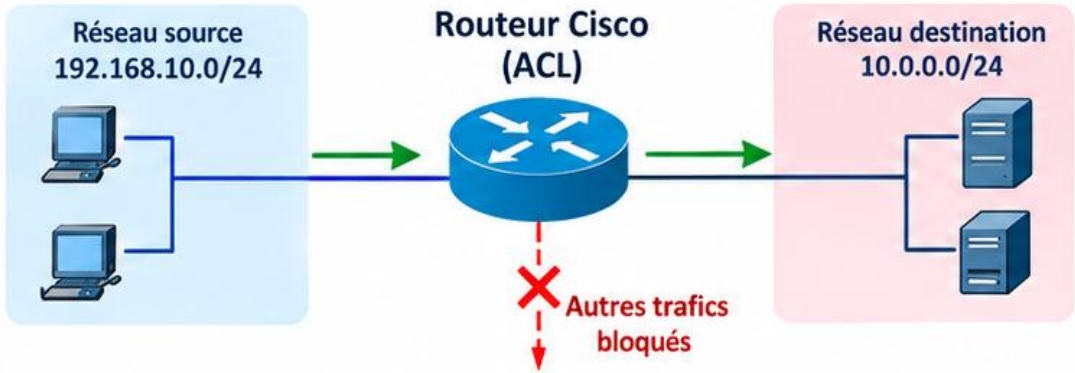
Contrôle d'accès (ACL) : Réseau classique (Cisco) vs SDN

Réseau classique (Cisco)

Configuration ACL sur un routeur/switch Cisco

```
! Autoriser le trafic HTTP du réseau 192.168.10.0/24
! vers le réseau 10.0.0.0/24

ip access-list extended WEB
  permit tcp 192.168.10.0 0.0.0.255 10.0.0.0 0.0.0.255 eq 80
  deny ip any any
!
interface GigabitEthernet0/1
  ip access-group WEB in
```



- Configuration manuelle sur chaque équipement
- Règles locales (par routeur/switch)
- Gestion complexe à grande échelle

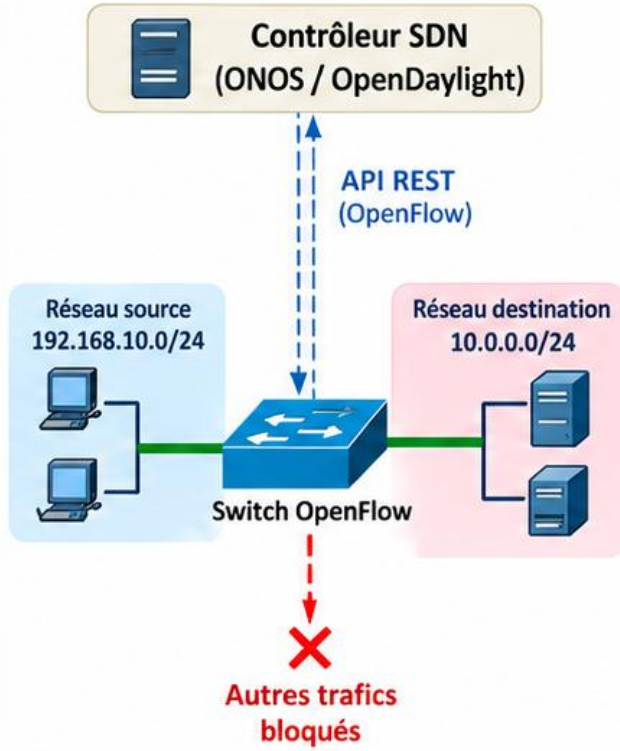
SDN (OpenFlow / Contrôleur)

Configuration via API (exemple Python)

```python
import requests
controller = "http://10.0.0.1:8080"
headers = {"Content-Type": "application/json"}

# Règle d'autorisation (HTTP)
flow = {
    "priority": 100,
    "match": {
        "eth_type": 0x0800,
        "ip_proto": 6,
        "ipv4_src": "192.168.10.0/24",
        "ipv4_dst": "10.0.0.0/24",
        "tcp_dst": 80
    },
    "actions": ["OUTPUT:port2"]
}
requests.post(f"{controller}/stats/flowentry/add",
              json=flow, headers=headers)

# Règle de blocage (tout le reste)
flow_deny = {
    "priority": 1,
    "match": {"eth_type": 0x0800},
    "actions": []  # drop
}
requests.post(f"{controller}/stats/flowentry/add",
              json=flow_deny, headers=headers)
```



- Politique centralisée via un contrôleur
- Règles installées dynamiquement sur les switches (OpenFlow)
- Gestion et modification simples
- Cohérence des politiques sur tout le réseau

